

ina M.

re pour connecter les grimpeurs, les clubs, les lieux et les

Événements clubs

RNCP39583 · BLOC 2

1
OBJETS DÉCLARÉS

Concevoir et développer des applications logicielles

Spity · Prototype web pour la communauté escalade

Lieu

Profil actif

lina.demo@spity.fr

Disciplines

bloc vole

Matériel

1 x Grand Rel

Candidat

Dorian Joly

Version

Spity 0.1.0

Livrable

Dossier BC02 et documentation d'exploitation

Date

20 juillet 2026

Il y a 18 min

Événement

Prochains ren

Sortie fal

Club Alpin

e cherche quelqu'un pour travailler les profils déversants et filmer

Sommaire des livrables

Le dossier commence par une synthèse et l'index exhaustif de la grille. Les chapitres suivants conservent le détail reproductible de chaque compétence.

01 Dossier de validation BC02 - Spity

02 Index des critères et des preuves BC02

03 Périmètre fonctionnel et user stories du prototype BC02

04 Environnements, qualité et protocole de déploiement - C2.1.1

05 Protocole d'intégration continue - C2.1.2

06 Harnais de tests unitaires - C2.2.2

07 Architecture et prototype maintenable - C2.2.1

08 Tests d'intégration du prototype - C2.2.2

09 Mesures de sécurité et matrice OWASP Top 10 - C2.2.3

10 Accessibilité du prototype et audit RGAA 4.1.2 - C2.2.3

11 Gestion des versions et déploiements progressifs - C2.2.4

12 Cahier de recettes - C2.3.1

13 Plan de correction des bogues - C2.3.2

14 Manuel de déploiement de Spity - C2.4.1

15 Manuel d'utilisation de Spity - C2.4.1

16 Manuel de mise à jour et de maintenance de Spity - C2.4.1

Dossier de validation BC02 - Spity

Identification

Champ	Valeur
Candidat	Dorian Joly
Certification	Expert en développement logiciel - RNCP39583
Bloc	BC02 - Concevoir et développer des applications logicielles
Projet	Spity, réseau social pour la communauté escalade
Prototype évalué	Application web responsive, version <code>0.1.0</code>
Branche de travail	<code>develop</code>
Release stable	<code>v0.1.0</code>
Date du dossier	20 juillet 2026
Dépôt	github.com/Dorianyloj/spity

Déclaration de périmètre

Ce dossier démontre les neuf compétences du bloc BC02 à partir d'une réalisation fonctionnelle, versionnée et testée. Il distingue volontairement :

- la vision produit complète de Spity ;
- le prototype BC02 effectivement développé ;
- les fonctions testées et démontrables ;
- les limites et évolutions encore nécessaires.

Une fonction absente du périmètre ou non persistante n'est pas présentée comme terminée. Les preuves associent systématiquement une explication, un fichier ou artefact, un résultat vérifié et une limite éventuelle.

Résumé du projet

Spity répond au besoin des grimpeurs qui utilisent plusieurs services distincts pour rechercher un partenaire, trouver un lieu ou participer à une sortie. Le prototype réunit deux rôles :

- le **grimpeur**, qui gère son profil et son matériel, recherche des partenaires, traite des demandes et s'inscrit à des événements ;
- le **club**, qui gère sa fiche, publie des événements et suit les participants.

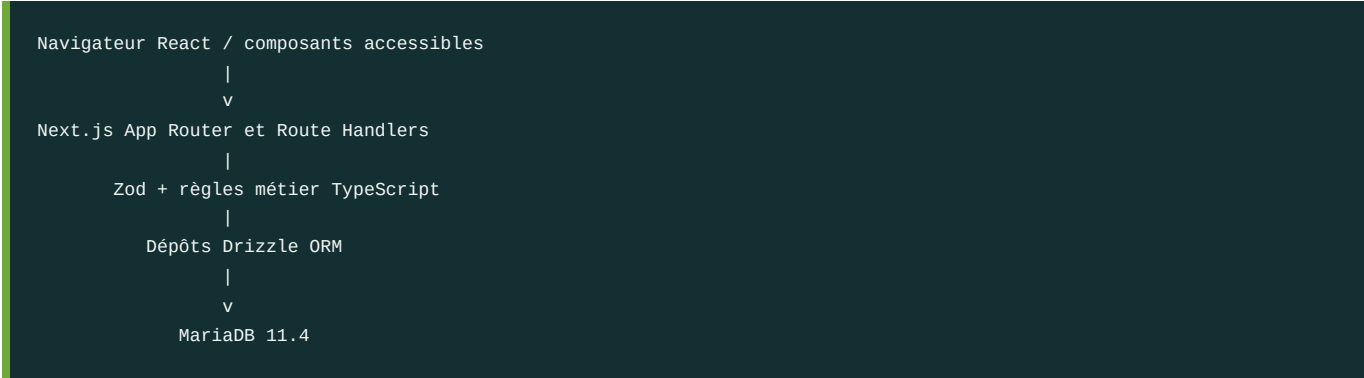
Le visiteur peut créer un compte ou se connecter. Les autorisations, la capacité des événements et les données visibles dépendent ensuite du rôle et de la session.

Périmètre démontrable

ID	Fonction
F01	Inscription, connexion, session et déconnexion.
F02	Création, consultation et modification des profils grimpeur et club.
F03	Recherche de partenaires par texte, discipline, niveau, disponibilité et environnement.
F04	Envoi, acceptation, refus et historique des demandes de partenariat.
F05	Création, modification et annulation d'un événement par son club.
F06	Consultation des événements et des capacités restantes.
F07	Inscription et désinscription d'un grimpeur.
F08	Consultation des participants par le club organisateur.
F09	Validation, contrôle de session, rôle, origine, quota et réponses sûres.
F10	Structure sémantique, clavier, reflow mobile et mouvement réduit.

Le fil social persistant, les likes/commentaires, la carte interactive, les topos collaboratifs, les notifications, la récupération de mot de passe et les parcours RGPD autonomes restent hors du prototype évalué.

Architecture synthétique



L'application utilise TypeScript strict de bout en bout. Next.js rassemble pages, composants serveur et API dans un artefact standalone Node.js 22. Zod valide les données à l'exécution, Drizzle structure les accès et les migrations, MariaDB conserve les relations métier. Docker sépare l'application, les migrations et la base. GitHub Actions construit et teste chaque révision avant staging.

Résultats vérifiés

Axe	Résultat de référence
Analyse statique	ESLint et TypeScript réussis.
Tests unitaires	86 tests réussis.
Couverture ciblée	96 % lignes, 89,69 % branches, 100 % fonctions.

Axe	Résultat de référence
Intégration	11 résultats HTTP/MariaDB réussis sur une base migrée à neuf.
Recette	6 scénarios Playwright couvrant F01 à F10, sans retry ni skip.
Sécurité	0 vulnérabilité haute ou critique de production ; dix catégories OWASP analysées.
Accessibilité authentifiée	Dix états à 100 % dans l'audit automatisé.
Lighthouse public	Performance 97 à 99, accessibilité 100, SEO 100.
Responsive	Reflow vérifié à 360 px sans défilement horizontal incohérent.
Livraison	Release <code>v0.1.0</code> , images immuables, bundle, manifeste et SHA-256.
Dernière CI probante	Run <code>29750556481</code> , cinq jobs réussis dont staging.

Couverture des compétences

C2.1.1 - Environnements et déploiement

Les environnements développement, test, staging et production sont séparés. Les outils, secrets, seuils et séquences de déploiement sont définis. Le staging construit des images par SHA uniquement après les contrôles qualité, puis vérifie migrations et santé dans un environnement isolé.

Preuve principale : [environnements, qualité et déploiement](#).

C2.1.2 - Intégration continue

Le pipeline décrit l'installation verrouillée, le lint, le typage, Jest, l'audit, les configurations Compose, le build, MariaDB, l'accessibilité, Lighthouse et la recette standalone. Un échec bloque la promotion, comme le prouve le run `29749715001` .

Preuve principale : [protocole d'intégration continue](#).

C2.2.1 - Prototype maintenable

L'architecture par feature sépare présentation, API, règles métier et données. Les composants partagés et les contrats Zod limitent la duplication. Le prototype répond aux user stories retenues sur desktop et mobile.

Preuve principale : [architecture et prototype](#).

C2.2.2 - Harnais unitaire

Jest couvre les validateurs, le parseur de matériel, les règles d'événements et de matching, le quota, le contrôle d'origine, les en-têtes et des composants critiques. L'intégration réelle et Playwright complètent les limites du périmètre unitaire.

Preuves principales : [harnais Jest](#) et [tests d'intégration](#).

C2.2.3 - Sécurité et accessibilité

La sécurité est reliée aux dix catégories OWASP 2025. Le RGAA 4.1.2 est retenu et justifié pour le contexte français. Les résultats automatisés sont complétés par des contrôles clavier, focus, mouvement réduit et mobile.

Preuves principales : [sécurité OWASP](#) et [accessibilité RGAA](#).

C2.2.4 - Versions et déploiements

Git, Semantic Versioning, le changelog et les images par SHA relient le besoin à l'artefact. La release stable promeut le même candidat testé et fournit un bundle de déploiement autonome.

Preuve principale : [versions et déploiements](#).

C2.3.1 - Cahier de recettes

Le cahier couvre toutes les fonctions F01 à F10, les rôles, les cas alternatifs, la sécurité, la structure et le mobile. Les scénarios sont automatisés et exécutés sur une MariaDB migrée.

Preuve principale : [cahier de recettes](#).

C2.3.2 - Correction des bogues

Sept anomalies réelles ont été détectées, qualifiées, corrigées et retestées. Le registre différencie les défauts produit des erreurs de harnais. Aucun seuil ni cookie de production n'a été affaibli pour obtenir une CI verte.

Preuve principale : [plan de correction](#).

C2.4.1 - Documentation d'exploitation

Trois manuels séparés permettent de déployer, utiliser et maintenir Spity. Ils décrivent les langages, technologies, commandes, rôles, diagnostics, sauvegardes, mises à jour et retours arrière.

Preuves principales : [manuel de déploiement](#), [manuel d'utilisation](#) et [manuel de maintenance](#).

Démonstration conseillée

1. Présenter le périmètre F01-F10 et les limites.
2. Se connecter avec le compte grimpeur Lina.
3. Montrer son profil, son matériel, les filtres de matching et les lieux.
4. Se connecter avec le compte club et créer un événement de capacité **1**.
5. Revenir au compte Lina et s'inscrire.
6. Revenir au club pour afficher la participante et annuler l'événement.
7. Présenter le run CI vert, le run bloqué et le registre de correction.
8. Ouvrir l'index des critères pour répondre aux questions du jury.

Limites et perspectives

La réalisation est un prototype de certification, pas une plateforme publique prête à accueillir des données réelles. Les limites principales sont :

- aucune session pilote externe formalisée à la date du dossier ;
- récupération de mot de passe, export et suppression autonome de compte non exposés ;
- fil social, carte interactive et topos encore démonstratifs ou hors périmètre ;
- alertes modérées de dépendances suivies, sans vulnérabilité haute ou critique ;
- supervision, sauvegardes planifiées et test de restauration à industrialiser avant production publique.

Les priorités suivantes sont la validation utilisateur, les parcours RGPD, la récupération de compte, la persistance sociale, la cartographie et le renforcement de la couverture des composants de présentation.

Conclusion

Spity démontre une chaîne complète de conception et de développement : besoin cadré, architecture structurée, fonctionnalités cohérentes, contrôles automatisés, correction d'anomalies, version stable et documentation d'exploitation. Le dossier ne repose pas uniquement sur une description : chaque compétence renvoie à du code versionné, une commande ou un résultat distant.

L'[index des critères](#) constitue le point d'entrée pour l'évaluation. Les chapitres détaillés qui suivent fournissent les explications et preuves nécessaires à chaque ligne de la grille.

Index des critères et des preuves BC02

1. Objet

Cet index reprend chaque critère de la grille d'évaluation du bloc 2 du titre Expert en développement logiciel RNCP39583. Il indique où le jury peut trouver l'explication, la réalisation technique et le résultat vérifié.

Sources officielles utilisées :

- Référentiel Expert en développement logiciel RNCP39583, bloc 2, pages 7 à 10 ;
- grille d'évaluation BC02 datée du 10 octobre 2024.

Statuts : **couvert** signifie que l'explication, la réalisation et une vérification sont présentes ; **couvert avec limite** signale une preuve complète sur le périmètre déclaré mais une extension encore possible.

2. Résultat global

Compétence	Livrable	Statut	Preuve principale
C2.1.1	Protocole de déploiement continu et critères qualité/performance	Couvert	Environnements et déploiement
C2.1.2	Protocole d'intégration continue	Couvert	Protocole CI
C2.2.1	Architecture maintenable et prototype fonctionnel	Couvert	Architecture du prototype
C2.2.2	Jeu de tests unitaires	Couvert avec limite	Harnais unitaire et intégration
C2.2.3	Sécurité et accessibilité	Couvert	OWASP et RGAA
C2.2.4	Historique et dernière version fiable	Couvert avec limite	Versions et déploiements
C2.3.1	Cahier de recettes	Couvert	Cahier F01 à F10
C2.3.2	Plan de correction des bogues	Couvert	Registre des anomalies
C2.4.1	Trois manuels d'exploitation	Couvert	Déploiement , utilisation , maintenance

3. C2.1.1 - Environnements, qualité et déploiement

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le protocole de déploiement continu est explicité.	Séquence de validation, staging immuable, migration, santé, promotion et retour arrière décrite dans les sections 8 et 9.	Document C2.1.1 , workflow CI , workflow release	Couvert
L'environnement de développement est détaillé.	Node.js 22, npm 10, Next.js, TypeScript, MariaDB, Docker Compose, variables et commandes de démarrage sont précisés.	Document C2.1.1 , package.json , Compose local	Couvert
Les outils permettent d'identifier compilateur, serveur d'application et gestion de sources.	TypeScript est vérifié par <code>tsc</code> , Next.js/Node fournit le serveur, Git et GitHub assurent les sources ; Drizzle, Jest, Playwright, Lighthouse et Docker complètent la chaîne.	Table des outils , README	Couvert
Le protocole définit les différentes séquences de déploiement.	Conditions d'entrée, sauvegarde, migration one-shot, démarrage, contrôle, promotion et rollback sont ordonnés.	Manuel de déploiement , runbook de release	Couvert

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les critères qualité/performance répondent aux exigences du projet.	Seuils lint, typage, couverture, vulnérabilités, accessibilité et Lighthouse sont bloquants.	Seuils C2.1.1 , run CI 29750556481, rapports locaux <code>.lighthouseci</code> et <code>coverage</code>	Couvert

4. C2.1.2 - Intégration continue

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le protocole d'intégration continue est explicité clairement.	Déclencheurs, environnement, permissions, stratégie de branches, revue et traitement d'échec sont documentés.	Protocole CI	Couvert
Le protocole définit les séquences d'intégration.	Les jobs qualité, MariaDB/accessibilité, Lighthouse et recette standalone précèdent obligatoirement le staging.	Workflow CI , run réussi 29750556481	Couvert

5. C2.2.1 - Architecture et prototype

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les bonnes pratiques de développement sont respectées.	Architecture par feature, TypeScript strict, composants serveur, validation Zod, dépôts Drizzle et composants UI partagés.	Architecture , src/features , src/components/ui	Couvert
Le prototype est fonctionnel et répond aux besoins identifiés.	Les rôles grimpeur et club réalisent inscription, profil, matching, partenariats et événements.	Périmètre et user stories , recette	Couvert
Le prototype met en œuvre un ensemble cohérent de fonctionnalités et les user stories.	F01 à F10 sont reliées à dix user stories et six scénarios navigateur.	Traçabilité fonctionnelle , architecture section 5	Couvert
Les composants de l'interface sont présents et fonctionnels.	Formulaires, filtres, onglets, boutons, navigation responsive, états vides et retours d'action sont pilotés par Playwright.	Captures visuelles , tests/acceptance	Couvert
Le prototype satisfait les exigences de sécurité.	Sessions HttpOnly, contrôle d'origine, autorisations par rôle, Zod, bcrypt, rate limiting et en-têtes de sécurité sont actifs et testés.	Matrice OWASP , scénario REC-F09-001	Couvert

6. C2.2.2 - Tests unitaires

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les tests unitaires couvrent la majorité du code développé.	Le périmètre unitaire déclaré atteint 96 % des lignes, 89,69 % des branches et 100 % des fonctions ; les routes et contrats inter-modules sont complétés par 11 résultats d'intégration et la recette F01-F10.	Harnais et périmètre , tests d'intégration , <code>coverage/coverage-summary.json</code>	Couvert avec limite

Limite déclarée : le pourcentage Jest porte sur les modules métier et de sécurité explicitement inclus dans `collectCoverageFrom`, pas sur chaque composant de présentation. Les parcours non unitaires sont couverts par intégration et Playwright ; l'extension de la couverture composant par composant reste possible.

7. C2.2.3 - Sécurité et accessibilité

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les mesures couvrent les dix principales failles OWASP.	La matrice OWASP Top 10:2025 associe chaque risque à une mesure, un fichier, un test et un risque résiduel.	Matrice OWASP , <code>npm run security:audit</code> , scénario REC-F09-001	Couvert
Le référentiel d'accessibilité est présenté et justifié.	Le RGAA 4.1.2 est retenu pour le contexte français et complété par des tests axe, Lighthouse et manuels.	Audit RGAA	Couvert
Le prototype répond aux exigences du référentiel établi.	Dix états authentifiés et trois pages publiques obtiennent 100 % en accessibilité automatisée ; clavier, lien d'évitement, mouvement réduit et reflow 360 px sont vérifiés.	Résultats RGAA , <code>.accessibility/summary.js</code> on , captures mobiles	Couvert

8. C2.2.4 - Versions et version fiable

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Un système de gestion de versions est utilisé.	Git, branches <code>develop / main</code> , Conventional Commits, SemVer, tags et images par SHA sont utilisés.	Gestion des versions , CHANGELOG.md , historique Git	Couvert
Les évolutions du prototype sont tracées.	Le changelog, les commits, les migrations et le registre d'anomalies relie chaque évolution à une preuve.	CHANGELOG.md , drizzle , bogues	Couvert
Le logiciel est fonctionnel et manipulable en autonomie.	Une release <code>v0.1.0</code> , un bundle, trois manuels et des comptes de démonstration permettent le démarrage et les parcours sans assistance au code.	Release v0.1.0 , manuel utilisateur	Couvert avec limite

Limite déclarée : la recette automatisée et la manipulation par le développeur sont prouvées. Une session pilote formalisée avec des utilisateurs externes au projet reste recommandée avant une exploitation publique.

9. C2.3.1 - Cahier de recettes

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Le cahier reprend l'ensemble des fonctionnalités attendues.	Les fonctions F01 à F10 sont toutes reliées à un scénario, des préconditions, des actions et des résultats.	Cahier de recettes , bc02-recipe.spec.ts	Couvert
Les tests fonctionnels, structurels et de sécurité sont conformes au plan.	Six scénarios Playwright vérifient rôles, données, capacité, sécurité, clavier et mobile sur MariaDB migrée.	Run 29750556481 , artefact <code>acceptance-689e59d...</code> , résultats	Couvert

10. C2.3.2 - Correction des bogues

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les bogues sont détectés, qualifiés et traités.	Sept anomalies réelles comportent impact, sévérité, priorité, cause, correction, SHA et retest.	Registre des anomalies	Couvert

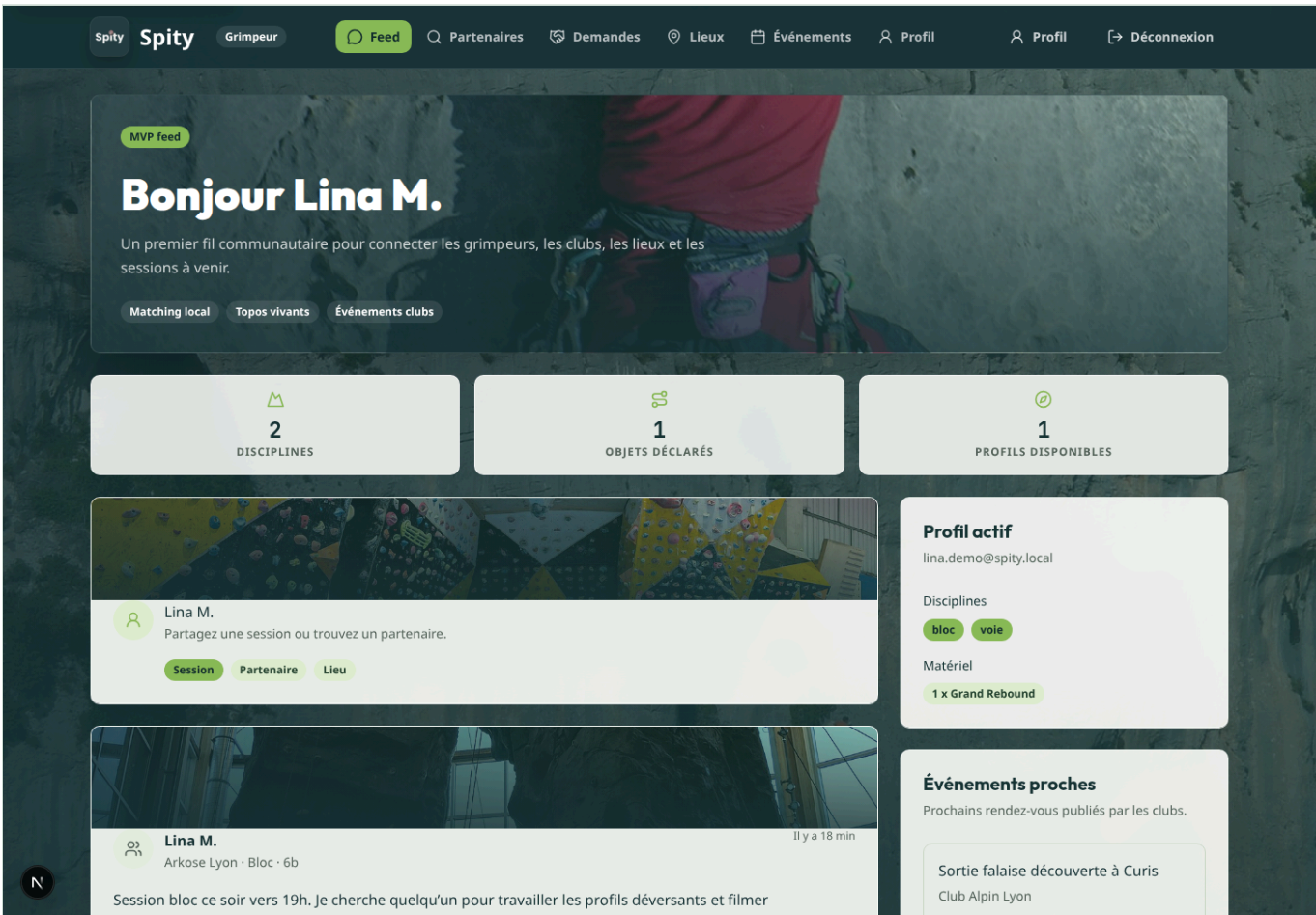
Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Une analyse des améliorations est réalisée pour chaque test en échec.	Les erreurs produit et les défauts du harnais sont distingués ; chaque échec Playwright observé possède une prévention.	Analyse des échecs	Couvert
Les corrections garantissent le bon fonctionnement attendu.	Aucun seuil ni exigence n'a été abaissé ; la CI suivante et le staging doivent être verts pour fermer une anomalie bloquante.	Runs <code>29749715001</code> bloqué puis <code>29750556481</code> réussi, commits <code>2941a44</code> et <code>689e59d</code>	Couvert

11. C2.4.1 - Documentation d'exploitation

Critère officiel	Explication et réalisation	Preuve vérifiable	Statut
Les manuels sont rédigés avec clarté.	Trois documents séparés s'adressent respectivement à l'exploitant, l'utilisateur et le mainteneur ; commandes, rôles, diagnostics et checklists sont explicités.	Déploiement , utilisation , mise à jour	Couvert
La documentation décrit les choix de technologies et de langages.	TypeScript, Next.js, React, Zod, Drizzle, MariaDB, Node.js, Docker, GitHub Actions et les outils de tests sont justifiés par couche.	Choix technologiques , repères de maintenance	Couvert

12. Preuves visuelles

Les captures suivantes sont générées par `npm run docs:capture` depuis les comptes de démonstration locaux :



Spity

Grimpeur

Feed

Partenaires

Demandes

Lieux

Événements

Profil

Profil

Déconnexion

Matching grimpeurs

Trouver un partenaire

Filtrez les profils disponibles puis envoyez une demande de partenariat suivie dans Spity.

Suivre mes demandes

1

PROFILS DISPONIBLES

1

RÉSULTATS

Nom ou localisation

Discipline

Niveau

Disponibilité

Environnement

Q Lyon, Camille...

Toutes les pratiques

Tous les niveaux

Toutes les disponibilités

Tous les environnements

Réinitialiser les filtres

N

Nassim B.

6a+

Villeurbanne

Voie et grande voie, disponible sur les pauses midi et le week-end.

voie trad mixed

Demande en attente

Spity

Club

Feed

Lieux

Événements

Profil

Profil

Déconnexion

Agenda communautaire

Événements Spity

Publiez vos rendez-vous, ajustez la capacité et consultez les participants.

Nouvel événement

2

ÉVÉNEMENTS VISIBLES

2

ÉVÉNEMENTS DU CLUB

Sortie

Sortie falaise découverte à Curis

7

place(s) disponible(s)

Club Alpin Lyon

27 juillet 2026 à 09:30

Curis-au-Mont-d'Or

1 / 8 participant(s)

Sortie encadrée pour découvrir la falaise, niveau conseillé 5c et matériel personnel vérifié.

Participants

Lina M.

Modifier Annuler l'événement

Contest

Contest bloc local Spity Crew

23

place(s) disponible(s)

Club Alpin Lyon

3 août 2026 à 18:30

Arkose Lyon

1 / 24 participant(s)

Contest amical ouvert à tous les niveaux, finale conviviale et lots partenaires.

Participants

Nassim B.

Modifier Annuler l'événement



13. Preuves distantes pérennes

Preuve	URL	Résultat
CI initiale complète de recette	Run 29747713909	Quatre portes et staging réussis.
Blocage réel du staging	Run 29749715001	Recette en échec, staging ignoré.
Retest standalone	Run 29750556481	Cinq jobs réussis et images publiées.
Release stable	Spity v0.1.0	Bundle, manifeste, somme SHA-256 et images versionnées.

14. Points restant à organiser

Ces points ne remettent pas en cause les critères techniques démontrés, mais doivent être présentés honnêtement au jury :

- conduire une session pilote formalisée avec au moins deux utilisateurs externes au développement ;
- conserver des captures des protections de branches GitHub si elles sont activées dans l'interface distante ;
- exécuter périodiquement un exercice de restauration sur un environnement isolé ;

- étendre la couverture Jest aux composants de présentation qui évoluent le plus ;
- poursuivre les fonctions hors prototype : carte interactive, fil social persistant, topos et parcours RGPD autonomes.

Périmètre fonctionnel et user stories du prototype BC02

1. Finalité du document

Ce document fixe le périmètre du prototype Spity présenté pour le bloc BC02 du titre Expert en développement logiciel RNCP39583.

Il constitue le contrat de référence entre :

- les besoins fonctionnels retenus ;
- les fonctionnalités à développer ;
- les tests unitaires et d'intégration ;
- le cahier de recettes ;
- les preuves présentées au jury.

Une fonctionnalité absente de ce périmètre ne sera pas présentée comme terminée. Toute évolution du périmètre devra être tracée dans l'historique du projet et répercutée dans les scénarios de recette.

2. Positionnement de Spity

Spity est une application web communautaire destinée aux pratiquants et aux clubs d'escalade. Le prototype BC02 doit démontrer un parcours cohérent permettant :

1. de créer et sécuriser un compte grimpeur ou club ;
2. de compléter un profil adapté au rôle choisi ;
3. de rechercher des partenaires compatibles ;
4. de demander et confirmer une mise en relation ;
5. de publier, consulter et gérer des événements de club ;
6. de s'inscrire à un événement dans la limite des places disponibles.

Le prototype est une application web responsive. Une application mobile native n'est pas incluse dans le périmètre évalué.

3. Acteurs

Visiteur

Utilisateur non authentifié pouvant consulter la présentation de Spity, créer un compte ou se connecter.

Grimpeur

Utilisateur authentifié possédant un profil grimpeur. Il peut gérer ses informations, rechercher des partenaires, gérer ses demandes de mise en relation, consulter les événements et s'y inscrire.

Club

Utilisateur authentifié possédant un profil club. Il peut gérer son profil, créer et administrer ses événements et consulter les inscriptions associées.

Systeme

Ensemble des traitements automatiques chargés de valider les données, contrôler les autorisations, maintenir les capacités des événements et produire des réponses cohérentes en cas d'erreur.

4. Périmètre retenu

Fonctionnalités obligatoires

Identifiant	Domaine	Fonctionnalité
F01	Authentification	Inscription avec choix du rôle, connexion, session et déconnexion.
F02	Profils	Création, consultation et modification du profil correspondant au rôle.
F03	Matching	Annuaire de grimpeurs avec filtres par discipline, niveau et localité.
F04	Mise en relation	Envoi, acceptation et refus d'une demande de partenariat.
F05	Événements	Création, modification et annulation d'un événement par un club.
F06	Découverte	Consultation des événements à venir et de leur capacité disponible.
F07	Inscriptions	Inscription et désinscription d'un grimpeur à un événement.
F08	Suivi club	Consultation de la liste et du nombre d'inscrits par le club organisateur.
F09	Sécurité	Contrôle des accès, validation des entrées, protection des sessions et gestion sûre des erreurs.
F10	Accessibilité	Parcours principaux utilisables au clavier, structurés et compréhensibles avec les technologies d'assistance.

Hors périmètre du prototype BC02

Les fonctionnalités suivantes appartiennent à la vision produit, mais ne sont pas nécessaires à la validation du prototype retenu :

- fil social, publications, médias, commentaires, likes et stories ;
- topos collaboratifs, secteurs, voies et votes de cotation ;
- carte géographique interactive et calcul de distance GPS ;
- messagerie instantanée et notifications push ;
- paiement, abonnement ou billetterie ;
- modération communautaire avancée ;
- validation automatique de l'affiliation auprès de la FFME ;
- application mobile native ;
- authentification par fournisseur externe ou OTP ;
- récupération de mot de passe par courrier électronique.

Ces exclusions devront apparaître dans le dossier final afin de distinguer clairement le prototype évalué des évolutions prévues.

5. Matrice simplifiée des autorisations

Action	Visiteur	Grimpeur	Club
Consulter la landing page	Oui	Oui	Oui
Créer un compte ou se connecter	Oui	Sans objet	Sans objet
Modifier son propre profil	Non	Oui	Oui
Rechercher des grimpeurs	Non	Oui	Non
Envoyer ou traiter une demande de partenariat	Non	Oui	Non
Consulter les événements	Non	Oui	Oui
Créer et administrer un événement	Non	Non	Oui
S'inscrire à un événement	Non	Oui	Non
Consulter les inscrits d'un événement	Non	Non	Oui, pour ses événements uniquement

L'absence d'autorisation doit conduire à une réponse explicite **401** pour un utilisateur non authentifié ou **403** pour un utilisateur authentifié ne disposant pas du rôle requis.

6. User stories et critères d'acceptation

US-AUTH-01 - Créer un compte

En tant que visiteur, **je veux** créer un compte grimpeur ou club **afin de** rejoindre Spity avec les droits correspondant à mon rôle.

Priorité : Must have.

Critères d'acceptation :

1. Étant donné une adresse électronique non utilisée, un mot de passe conforme et un rôle valide, lorsque le formulaire est soumis, alors le compte est créé, une session sécurisée est ouverte et l'utilisateur est dirigé vers l'onboarding.
2. Une adresse électronique est normalisée et ne peut être associée qu'à un seul compte.
3. Un mot de passe doit contenir au moins huit caractères, une majuscule, une minuscule, un chiffre et un caractère spécial.
4. Une adresse déjà utilisée produit une erreur métier explicite sans créer de doublon.
5. Une donnée invalide est refusée côté client et côté serveur.
6. Les erreurs sont annoncées de manière accessible et ne reposent pas uniquement sur la couleur.

US-AUTH-02 - Se connecter et se déconnecter

En tant que titulaire d'un compte, **je veux** ouvrir et fermer une session **afin de** protéger l'accès à mes données.

Priorité : Must have.

Critères d'acceptation :

1. Des identifiants valides ouvrent une session et redirigent vers le profil ou l'onboarding selon l'état du compte.
2. Des identifiants invalides retournent un message générique ne révélant pas l'existence du compte.
3. Le cookie de session est **HttpOnly**, **SameSite** et **Secure** en production.

4. Les routes protégées refusent les requêtes sans session valide.
5. La déconnexion invalide le cookie de session et interdit immédiatement l'accès aux routes protégées depuis le navigateur.
6. Les tentatives anormales sont limitées sans permettre le verrouillage abusif d'un compte par un tiers.

US-PROFILE-01 - Gérer un profil grimpeur

En tant que grimpeur, **je veux** renseigner mon profil **afin de** recevoir des résultats de matching pertinents.

Priorité : Must have.

Critères d'acceptation :

1. Le profil comporte un nom d'affichage, une bio facultative, une localité, au moins une discipline, le niveau correspondant à chaque discipline sélectionnée et le matériel disponible.
2. Les disciplines proposées sont au minimum le bloc, la voie et le trad.
3. Les cotations acceptées vont de 4a à 8c+ pour le prototype.
4. Un utilisateur ne peut créer qu'un seul profil grimpeur.
5. Seul le propriétaire peut modifier son profil.
6. L'adresse électronique et les données de session ne sont jamais affichées dans l'annuaire public.

US-PROFILE-02 - Gérer un profil club

En tant que club, **je veux** renseigner mon identité et ma localisation **afin de** publier des événements identifiables.

Priorité : Must have.

Critères d'acceptation :

1. Le profil comporte un nom, une bio facultative, une localité et un numéro d'affiliation FFME facultatif.
2. Un utilisateur ne peut créer qu'un seul profil club.
3. Seul le propriétaire peut modifier le profil.
4. L'absence de vérification automatique FFME est indiquée comme limite du prototype ; aucun badge « vérifié » n'est attribué automatiquement.

US-MATCH-01 - Rechercher des partenaires

En tant que grimpeur, **je veux** filtrer les autres grimpeurs **afin de** trouver un partenaire compatible.

Priorité : Must have.

Critères d'acceptation :

1. L'annuaire n'affiche que les profils grimpeurs complétés et n'affiche pas le profil de l'utilisateur courant.
2. Les résultats peuvent être filtrés par discipline, niveau et localité.
3. Plusieurs filtres actifs sont combinés et leur état reste visible.
4. La suppression des filtres restaure la liste complète autorisée.
5. Une recherche sans résultat affiche un état vide compréhensible et permet de réinitialiser les filtres.
6. Les résultats suivent un ordre déterministe documenté.

US-MATCH-02 - Gérer une demande de partenariat

En tant que grimpeur, **je veux** proposer une mise en relation et répondre aux demandes reçues **afin de** confirmer un intérêt mutuel.

Priorité : Must have.

Critères d'acceptation :

1. Un grimpeur peut envoyer une demande à un autre grimpeur depuis sa fiche.
2. Il est impossible de s'envoyer une demande à soi-même.
3. Une seule demande active peut exister pour une même paire de grimpeurs.
4. Le destinataire peut accepter ou refuser une demande en attente.
5. Seul le destinataire peut traiter la demande.
6. Les statuts autorisés sont `pending`, `accepted` et `declined` ; toute transition invalide est refusée.
7. Les demandes émises et reçues sont consultables dans un tableau de bord.

US-EVENT-01 - Administrer un événement

En tant que club, **je veux** créer, modifier ou annuler un événement **afin de** proposer une activité aux grimpeurs.

Priorité : Must have.

Critères d'acceptation :

1. Un événement comporte un titre, un type, une description, une localité, une date et heure de début, une date et heure de fin et une capacité strictement positive.
2. La fin est postérieure au début et un nouvel événement commence dans le futur.
3. Seul un compte club possédant un profil complet peut créer un événement.
4. Seul le club organisateur peut modifier ou annuler son événement.
5. Une capacité ne peut pas être réduite sous le nombre d'inscriptions actives.
6. Un événement annulé reste traçable, est indiqué comme tel et n'accepte plus d'inscription.

US-EVENT-02 - Consulter les événements

En tant que utilisateur authentifié, **je veux** consulter les événements à venir **afin de** découvrir les activités proposées par les clubs.

Priorité : Must have.

Critères d'acceptation :

1. La liste présente les événements futurs non annulés dans l'ordre chronologique.
2. Chaque résultat affiche le club, le titre, le type, la localité, la date et le nombre de places restantes.
3. La fiche d'un événement fournit l'ensemble des informations utiles sans exposer les données privées des participants.
4. Un état vide est affiché lorsqu'aucun événement ne correspond aux filtres.

US-EVENT-03 - S'inscrire à un événement

En tant que grimpeur, **je veux** m'inscrire ou me désinscrire d'un événement **afin de** gérer ma participation.

Priorité : Must have.

Critères d'acceptation :

1. Seul un grimpeur possédant un profil complet peut s'inscrire.
2. Une seule inscription active est autorisée par couple utilisateur/événement.
3. Une inscription est refusée si l'événement est complet, annulé ou déjà commencé.
4. Deux inscriptions concurrentes ne peuvent pas dépasser la capacité définie.
5. Une désinscription avant le début libère immédiatement une place.
6. Le grimpeur peut consulter ses inscriptions à venir.

US-EVENT-04 - Suivre les inscriptions d'un club

En tant que club organisateur, **je veux** consulter les participants de mes événements **afin de** préparer l'activité et suivre la capacité.

Priorité : Must have.

Critères d'acceptation :

1. Le club voit le nombre d'inscrits, la capacité et les places restantes.
2. Le club voit uniquement le nom d'affichage et les informations de profil nécessaires à l'organisation.
3. Un club ne peut jamais consulter la liste privée d'un événement appartenant à un autre club.
4. La liste reflète immédiatement les inscriptions et désinscriptions confirmées.

7. Exigences non fonctionnelles

Sécurité

La référence retenue est l'[OWASP Top 10:2025](#), version publiée la plus récente au moment du cadrage.

Le prototype devra notamment assurer :

- le contrôle d'accès côté serveur pour chaque opération protégée ;
- la validation et la normalisation des entrées ;
- des requêtes paramétrées via l'ORM ;
- la protection des mots de passe et des secrets ;
- une configuration sûre des cookies et en-têtes HTTP ;
- la prévention des doublons et des transitions métier incohérentes par des contraintes de base de données ;
- une gestion contrôlée des erreurs sans fuite de données sensibles ;
- la surveillance des dépendances et de la chaîne de construction ;
- une journalisation exploitable des événements de sécurité sans journaliser de mot de passe ou de jeton.

Une matrice séparée reliera chacune des dix catégories OWASP 2025 aux mesures, tests et preuves de Spity.

Accessibilité

La référence retenue est le [RGAA 4.1.2](#), version officielle en vigueur au moment du cadrage. Le RGAA 5 est annoncé pour fin 2026 ; son arrivée sera surveillée sans bloquer les travaux fondés sur la version 4.1.2.

Les parcours de l'échantillon d'audit devront au minimum garantir :

- une structure de titres et des régions cohérentes ;
- des libellés, instructions et erreurs de formulaire explicitement associés ;
- une navigation complète au clavier avec focus visible ;
- un ordre de tabulation logique ;
- des contrastes conformes ;
- des composants utilisables sans dépendre uniquement de la couleur ou du mouvement ;
- le respect de la préférence de réduction des animations ;
- des messages dynamiques annoncés aux technologies d'assistance ;
- un affichage utilisable avec zoom et sur petit écran.

Qualité et tests

Les seuils initiaux du prototype sont :

- aucune erreur ESLint ;
- aucune erreur TypeScript ;
- build de production réussi ;
- tests unitaires et d'intégration réussis ;
- couverture globale minimale de 80 % pour les lignes, instructions et fonctions, et de 70 % pour les branches ;
- aucune vulnérabilité de dépendance critique ou haute non justifiée ;
- chaque anomalie détectée liée à une entrée du plan de correction.

Les exclusions de couverture devront être rares, justifiées et documentées.

Performance et compatibilité

- interface responsive de 360 à 1 440 pixels de largeur ;
- score Lighthouse cible d'au moins 85 en performance et 100 en accessibilité sur les pages principales ;
- temps de réponse cible inférieur à 500 ms au 95e percentile pour les routes API principales dans l'environnement de référence, hors latence réseau externe ;
- absence de requête non bornée sur les listes ;
- prise en charge des versions récentes de Chromium et Firefox ;
- états de chargement, vide, succès et erreur présents sur chaque parcours asynchrone.

8. Échantillon prévu pour l'audit et la recette

L'échantillon minimal comprend :

1. la landing page ;
2. l'inscription ;
3. la connexion ;
4. l'onboarding grimpeur ;
5. l'onboarding club ;
6. l'annuaire et les filtres de matching ;
7. le tableau de bord des demandes ;
8. la liste et la fiche d'un événement ;
9. la création/modification d'un événement ;
10. les inscriptions du grimpeur et le suivi des participants du club ;
11. les principaux cas d'erreur et d'accès interdit.

9. Traçabilité vers le BC02

Élément du présent cadrage	Compétences alimentées	Preuve attendue ultérieurement
Périmètre, acteurs et user stories	C2.2.1, C2.3.1	Présentation du prototype et cahier de recettes.
Critères d'acceptation	C2.2.2, C2.3.1	Tests automatisés et scénarios de recette.
Matrice des autorisations et exigences de sécurité	C2.2.3	Matrice OWASP, tests d'autorisation et rapport de sécurité.
Exigences RGAA	C2.2.3	Audit d'accessibilité, corrections et preuves visuelles.
Seuils qualité et performance	C2.1.1, C2.1.2	Pipeline CI, rapports de couverture et Lighthouse.

Élément du présent cadrage	Compétences alimentées	Preuve attendue ultérieurement
Périmètre versionné et exclusions	C2.2.4, C2.4.1	Historique des versions et documentation de maintenance.

Identifiants réservés pour le cahier de recettes

Fonctionnalité	User stories	Préfixe des scénarios de recette
F01 - Authentification	US-AUTH-01, US-AUTH-02	REC-AUTH
F02 - Profils	US-PROFILE-01, US-PROFILE-02	REC-PROFILE
F03 - Matching	US-MATCH-01	REC-MATCH
F04 - Mise en relation	US-MATCH-02	REC-PARTNER
F05 - Administration des événements	US-EVENT-01	REC-EVENT-ADMIN
F06 - Découverte des événements	US-EVENT-02	REC-EVENT-LIST
F07 - Inscriptions	US-EVENT-03	REC-REGISTRATION
F08 - Suivi club	US-EVENT-04	REC-EVENT-PARTICIPANTS
F09 - Sécurité	Exigence transverse	REC-SECURITY
F10 - Accessibilité	Exigence transverse	REC-A11Y

Chaque critère d'acceptation recevra un identifiant numéroté lors de la rédaction du cahier de recettes, par exemple **REC-AUTH-001**.

10. Définition de terminé d'une fonctionnalité

Une fonctionnalité n'est terminée que si :

1. ses critères d'acceptation sont satisfaits ;
2. les contrôles d'autorisation sont réalisés côté serveur ;
3. les données sont protégées par les contraintes nécessaires ;
4. les tests automatisés pertinents sont présents et réussissent ;
5. le parcours clavier et les erreurs accessibles ont été vérifiés ;
6. le scénario correspondant du cahier de recettes est rédigé ;
7. les choix et limites sont documentés ;
8. la fonctionnalité est intégrée par le pipeline sans régression.

11. Décision de cadrage

Le présent périmètre constitue la version de référence du prototype BC02. Les fonctions F01 à F10 sont obligatoires. Les fonctions placées hors périmètre ne conditionnent pas la complétude du prototype, mais peuvent être présentées comme perspectives après validation des neuf compétences du bloc.

Environnements, qualité et protocole de déploiement - C2.1.1

1. Objectif et compétence couverte

Ce document décrit la mise en œuvre des environnements de développement, de test et de production de Spity ainsi que les outils de suivi de qualité et de performance.

Il répond à la compétence C2.1.1 du bloc BC02 :

Mettre en œuvre des environnements de déploiement et de test en y intégrant les outils de suivi de performance et de qualité afin de permettre le bon déroulement de la phase de développement du logiciel.

Les livrables associés sont :

- le protocole de déploiement continu ;
- les critères de qualité et de performance ;
- la description de l'environnement de développement ;
- l'identification du compilateur, du serveur d'application et du gestionnaire de sources ;
- les séquences de déploiement et de retour arrière.

2. Environnement de développement

Poste de référence

Composant	Choix	Rôle
Système	Windows avec PowerShell	Poste de développement de référence.
Éditeur	Visual Studio Code ou Codex	Édition, revue et navigation dans le code.
Runtime	Node.js 22	Exécution de Next.js et des outils npm.
Gestionnaire de paquets	npm 10 ou supérieur	Installation reproductible via <code>npm ci</code> .
Framework	Next.js 16 avec React 19	Serveur d'application et interface web.
Compilateur	TypeScript 5 et compilateur Next.js/Turbopack	Typage statique et production des bundles.
Base de données	MariaDB 11.4 dans Docker	Persistance locale isolée du poste.
ORM	Drizzle ORM et Drizzle Kit	Requêtes typées et migrations.
Gestion de sources	Git et GitHub	Historique, branches, revues et intégration.

La version Node.js attendue est conservée dans `spity/.nvmrc`. Les contraintes compatibles sont également déclarées dans `package.json`.

Démarrage reproductible

Depuis le dossier `spity/` de l'application :


```
npm ci
cp .env.example .env.local
# Remplacer les secrets de démonstration.
docker compose --env-file .env.local up -d --wait
npm run db:migrate
npm run dev
```

La base est publiée uniquement sur `127.0.0.1:3306`. phpMyAdmin n'est pas démarré par défaut et nécessite le profil explicite `tools`.

3. Séparation des environnements

Propriété	Développement	Test	Production
Fichier modèle	<code>.env.example</code>	<code>.env.test.example</code>	<code>.env.production.example</code>
Fichier secret local	<code>.env.local</code>	<code>.env.test</code>	<code>.env.production</code>
Orchestration	<code>docker-compose.yml</code>	<code>docker-compose.test.yml</code>	<code>docker-compose.production.yml</code>
Base	MariaDB persistante	MariaDB isolée en <code>tmpfs</code>	MariaDB persistante, non exposée au réseau hôte
Port base	<code>127.0.0.1:3306</code>	<code>127.0.0.1:3307</code>	Aucun port publié
Application	<code>next dev</code> sur le poste	Serveur de test/CI	Image Docker Next.js standalone
Données	Données locales	Données jetables	Volume sauvegardé
Secrets	Valeurs locales non versionnées	Valeurs réservées aux tests	Secrets injectés au déploiement

Les fichiers contenant les valeurs réelles sont ignorés par Git. Seuls les modèles suffixés `.example` sont versionnés.

4. Architecture de l'environnement de production

Le fichier `docker-compose.production.yml` définit trois responsabilités :

- `mariadb` conserve les données dans un volume et n'expose aucun port à l'extérieur du réseau Docker ;
- `migrate` utilise la cible Docker `migration` pour appliquer les migrations avant le démarrage de la nouvelle version ;
- `app` exécute uniquement la sortie Next.js standalone, sans dépendances de développement.

L'application est liée à `127.0.0.1:3000` par défaut. En exploitation, un reverse proxy termine HTTPS et transmet les requêtes à ce port local.

La route `GET /api/health` vérifie réellement l'accès à MariaDB et identifie le binaire déployé. Elle retourne :

- `200 { "status": "ok", "version": "X.Y.Z", "revision": "SHA" }` lorsque l'application et la base sont disponibles ;
- `503 { "status": "unavailable", "version": "X.Y.Z", "revision": "SHA" }` lorsqu'une dépendance est indisponible.

Aucun détail de connexion ou message interne n'est exposé par cette route.

5. Gestion des secrets

Les règles suivantes sont obligatoires :

- aucun mot de passe, jeton ou secret réel dans Git, une image Docker ou un rapport ;
- secrets différents entre développement, test et production ;
- mot de passe applicatif MariaDB différent du mot de passe `root` ;
- secret de session généré aléatoirement avec au moins 64 octets en production ;
- rotation d'un secret immédiatement après toute exposition suspectée ;
- valeur `DATABASE_URL` cohérente avec l'utilisateur et le mot de passe MariaDB ;
- caractères spéciaux d'un mot de passe encodés lorsqu'ils sont placés dans une URL.

Commande de génération recommandée :

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

6. Outils de qualité et de performance

Outil	Commande	Résultat attendu
ESLint	<code>npm run lint</code>	Aucune erreur.
TypeScript	<code>npm run typecheck</code>	Aucune erreur de type.
Next.js	<code>npm run build</code>	Build de production réussi.
npm audit	<code>npm run security:audit</code>	Aucune vulnérabilité haute ou critique en production.
Lighthouse	<code>npm run perf:audit</code>	Respect des seuils des pages principales.
Docker Compose	<code>docker compose ... config --quiet</code>	Configuration valide pour les trois environnements.
Healthcheck	<code>GET /api/health</code>	Réponse HTTP 200 après déploiement.

Les tests et la couverture automatisée sont exécutés par `npm run test:coverage`. Lighthouse mesure les pages d'accueil, de connexion et d'inscription, vérifie chaque seuil et enregistre les rapports JSON dans `.lighthouseci`.

7. Critères de qualité et de performance

Indicateur	Seuil bloquant
Erreurs ESLint	0
Erreurs TypeScript	0
Build de production	Réussi
Tests automatisés	100 % des tests réussis
Couverture lignes, instructions et fonctions	Au moins 80 %
Couverture branches	Au moins 70 %

Indicateur	Seuil bloquant
Vulnérabilités critiques ou hautes de production	0 non justifiée
Lighthouse performance	Au moins 85/100
Lighthouse accessibilité	100/100
Lighthouse bonnes pratiques	Au moins 90/100
Lighthouse SEO	Au moins 90/100
API principales dans l'environnement de référence	95e percentile inférieur à 500 ms
Healthcheck après déploiement	HTTP 200

Les seuils Lighthouse sont codés dans `spity/scripts/run-lighthouse.mjs`. Une mesure reproductible est réalisée sur chaque page publique ; l'audit authentifié séparé vérifie dix états supplémentaires et le reflow mobile.

8. Protocole de déploiement continu

8.1 Conditions d'entrée

Un déploiement ne peut commencer que si :

1. la modification est tracée dans Git ;
2. la branche a été relue et intégrée selon le protocole CI ;
3. lint, typage, tests, audit et build sont réussis ;
4. la migration éventuelle a été relue et testée sur une base jetable ;
5. une version d'image immuable est identifiée par le SHA du commit ou un tag de version ;
6. la sauvegarde et la procédure de retour arrière sont prêtes.

8.2 Séquence de déploiement

1. Construire les images avec les cibles `runner` et `migration` du `Dockerfile`.
2. Étiqueter l'image avec le numéro de version et le SHA Git, sans réutiliser uniquement `latest` comme preuve.
3. Démarrer ou vérifier MariaDB et attendre son état `healthy`.
4. Sauvegarder la base avant toute migration de schéma en production.
5. Exécuter le service `migrate` une seule fois.
6. Démarrer la nouvelle version de `app` sur l'environnement de validation.
7. Attendre le healthcheck puis exécuter les smoke tests et le cahier de recettes ciblé.
8. Exécuter Lighthouse sur les pages représentatives.
9. Promouvoir les mêmes digests validés vers la production, sans reconstruction.
10. Vérifier le healthcheck, les journaux, les parcours critiques et les indicateurs de performance.
11. Enregistrer la version, la date, le SHA, l'opérateur et les résultats dans l'historique de déploiement.

Commandes de référence :

```
docker compose --env-file .env.production -f docker-compose.production.yml pull app migrate
docker compose --env-file .env.production -f docker-compose.production.yml up -d mariadb
docker compose --env-file .env.production -f docker-compose.production.yml --profile migration run --rm --no-build
migrate
docker compose --env-file .env.production -f docker-compose.production.yml up -d --no-build app
docker compose --env-file .env.production -f docker-compose.production.yml ps
```

8.3 Vérifications après déploiement

- `app` et `mariadb` sont `healthy` ;
- `/api/health` répond en HTTP 200 ;
- inscription, connexion et consultation du profil fonctionnent ;
- aucune erreur nouvelle n'apparaît dans les journaux ;
- les métriques Lighthouse respectent les seuils ;
- aucune migration n'a supprimé ou rendu incohérentes les données attendues.

8.4 Retour arrière

Le retour arrière est déclenché lorsqu'un contrôle bloquant échoue ou lorsqu'une anomalie critique est détectée.

1. Arrêter la promotion et conserver les journaux utiles au diagnostic.
2. Redéploier l'image immuable précédemment validée.
3. Restaurer la sauvegarde uniquement si la migration n'est pas compatible avec l'ancienne version.
4. Vérifier `/api/health` et les parcours critiques.
5. Créer une anomalie avec la cause, l'impact, les éléments de preuve et la décision prise.
6. Corriger sur une branche séparée puis reprendre la séquence complète.

Les migrations doivent être conçues de manière rétrocompatible lorsque cela est possible : ajout avant suppression, remplissage des nouvelles colonnes, bascule applicative, puis nettoyage dans une version ultérieure.

9. Éléments de preuve C2.1.1

Les preuves à conserver pour le dossier final sont :

- les trois configurations Compose et leurs fichiers d'exemple ;
- le `Dockerfile` multi-étapes et `.dockerignore` ;
- une capture d'une validation `docker compose config --quiet` réussie ;
- une capture des conteneurs `healthy` ;
- un résultat de `npm run quality` ;
- un rapport `npm run security:audit` ;
- les rapports Lighthouse générés dans `.lighthouseci` ;
- un exemple de déploiement réussi et un exercice de retour arrière ;
- l'historique Git des évolutions de l'environnement.

10. Résultats de validation initiaux

État vérifié le 20 juillet 2026 :

- les configurations Compose développement, test et production sont syntaxiquement valides ;

- l'audit des dépendances de production ne signale aucune vulnérabilité haute ou critique et conserve deux alertes modérées liées à `postcss` dans Next.js ;
- l'audit complet conserve six alertes modérées, dont la chaîne de développement `drizzle-kit/esbuild` ;
- le seuil CI est volontairement bloquant à partir du niveau `high` ; les alertes modérées restent suivies et ne sont pas masquées ;
- leur évolution reste suivie et une mise à jour non cassante devra être appliquée dès sa disponibilité.

L'image standalone a été construite sous Node.js 22, lancée sous l'utilisateur non privilégié `nextjs` et vérifiée avec une MariaDB de test migrée. La route `/api/health` a répondu `200 { "status": "ok" }`. Les résultats Lighthouse doivent être actualisés à chaque version livrée.

Résultats Lighthouse 12.6.1 obtenus sur le build de production :

Page	Performance	Accessibilité	Bonnes pratiques	SEO
Accueil	98	100	100	100
Connexion	97	100	100	100
Inscription	99	100	100	100

Le premier audit avait détecté une performance de 77 sur l'accueil et une accessibilité de 90 sur les formulaires. L'optimisation des images, la correction du contraste, de la hiérarchie des titres et de la taille des cibles tactiles ont permis de franchir les seuils.

Protocole d'intégration continue - C2.1.2

1. Objectif et compétence couverte

Ce document décrit le système d'intégration continue de Spity. Il répond à la compétence C2.1.2 du bloc BC02 : fusionner régulièrement le code source et tester les blocs de code afin de réduire les risques de régression.

La réalisation technique est versionnée dans `.github/workflows/ci.yml`.

La configuration suit les recommandations officielles :

- [construire et tester une application Node.js avec GitHub Actions](#) ;
- [configurer Jest avec Next.js](#).

2. Déclencheurs du pipeline

Le workflow `Continuous integration` est déclenché lors :

- d'un `push` vers `develop` ou `main` ;
- de l'ouverture ou de la mise à jour d'une pull request ciblant `develop` ou `main`.

Une seule exécution est conservée par branche. Lorsqu'un nouveau commit est poussé, l'exécution obsolète est annulée afin de ne pas produire de résultat portant sur une ancienne révision.

Le workflow dispose uniquement de l'autorisation `contents: read`. Il ne peut ni modifier le dépôt ni publier une version.

3. Environnement d'intégration

Élément	Configuration
Exécuteur	GitHub-hosted runner <code>ubuntu-latest</code>
Runtime	Version Node.js déclarée dans <code>spity/.nvmrc</code>
Gestionnaire	npm et installation déterministe avec <code>npm ci</code>
Verrou de dépendances	<code>spity/package-lock.json</code>
Cache	Cache npm indexé par le fichier de verrouillage
Dossier d'exécution	<code>spity/</code>
Durée maximale	20 minutes
Secrets de build	Valeurs de test non sensibles injectées dans le job

Le job `Quality gates` ne démarre pas MariaDB : ses tests sont unitaires et le build Next.js n'ouvre pas de connexion à la base. Un job indépendant `MariaDB integration tests` démarre toutefois une base vide, applique les migrations et vérifie les routes et dépôts du prototype.

4. Séquence d'intégration

Les étapes s'exécutent dans l'ordre suivant. Une étape en échec bloque toutes les étapes suivantes, sauf la publication du rapport de couverture configurée avec `if: always()`.

Ordre	Étape	Commande ou action	Critère de succès
1	Récupération des sources	<code>actions/checkout@v6</code>	SHA demandé disponible sur le runner
2	Installation de Node.js	<code>actions/setup-node@v6</code>	Version de <code>.nvmrc</code> active
3	Installation déterministe	<code>npm ci</code>	Dépendances conformes au lockfile
4	Analyse statique	<code>npm run lint</code>	Aucune erreur ou avertissement
5	Vérification du typage	<code>npm run typecheck</code>	Aucune erreur TypeScript
6	Tests et couverture	<code>npm run test:coverage</code>	Tous les tests et seuils réussis
7	Audit des dépendances	<code>npm run security:audit</code>	Aucune alerte haute ou critique de production
8	Validation de l'infrastructure	Trois commandes <code>docker compose ... config --quiet</code>	Configurations développement, test et production valides
9	Construction	<code>npm run build</code>	Build de production Next.js réussi
10	Conservation de la preuve	<code>actions/upload-artifact@v6</code>	Rapport <code>coverage-<SHA></code> conservé 30 jours

Job d'intégration MariaDB

Ordre	Étape	Critère de succès
1	Démarrer <code>mariadb:11.4</code>	Healthcheck MariaDB réussi
2	Installer avec <code>npm ci</code>	Lockfile respecté
3	Exécuter <code>npm run db:migrate</code>	Toutes les migrations s'appliquent sur une base vide
4	Exécuter <code>npm run test:integration</code>	11 résultats TAP réussis, aucune fuite de capacité, d'autorisation ou de quota d'authentification
5	Exécuter <code>npm run accessibility:audit</code>	Dix états authentifiés à 100 %, lien d'évitement, mouvement réduit et reflow mobile validés
6	Publier les artefacts	Rapports TAP, Lighthouse authentifié et captures mobiles conservés 30 jours

5. Protocole de contribution et de fusion

1. Mettre à jour `develop` avant de créer une branche courte `feat/...`, `fix/...`, `test/...` ou `docs/...`.
2. Réaliser un changement logique et l'accompagner des tests adaptés au risque.
3. Exécuter localement `npm run quality` avant le push.
4. Pousser la branche et ouvrir une pull request vers `develop`.
5. Vérifier que le périmètre, les migrations éventuelles et les risques de sécurité sont décrits dans la pull request.
6. Attendre le succès du contrôle `Quality gates`.

7. Faire relire le changement et traiter chaque commentaire bloquant.
8. Fusionner uniquement lorsque la revue et la CI sont validées.
9. Réserver `main` aux versions stables promues depuis `develop`.
10. Identifier chaque version livrée avec un tag et reporter son SHA dans le journal des versions.

Les protections de branches GitHub devront rendre le contrôle `Quality gates` et une revue obligatoires avant fusion vers `main`. Une capture de cette configuration sera conservée comme preuve externe au dépôt.

6. Traitement d'un échec

Lorsqu'une étape échoue :

1. la fusion ou la promotion est suspendue ;
2. le journal de l'étape permet d'identifier la commande et le fichier concernés ;
3. une anomalie est créée si l'échec révèle une régression ou un défaut du produit ;
4. la correction est réalisée sur la même branche ou sur une branche `fix/...` dédiée ;
5. le pipeline complet est rejoué depuis l'installation déterministe ;
6. aucun contrôle n'est désactivé pour rendre artificiellement la CI verte.

Une dépendance vulnérable ne peut être ignorée que si son périmètre, son exploitabilité, la mesure compensatoire, le responsable et la date de réévaluation sont documentés.

7. Résultats initiaux

Contrôles locaux exécutés le 20 juillet 2026 :

Contrôle	Résultat
ESLint	Succès, aucune erreur
TypeScript	Succès, aucune erreur
Jest	49 tests réussis sur 49
Couverture ciblée	97,5 % lignes, 87,91 % branches, 100 % fonctions
Audit de production au niveau <code>high</code>	Succès, aucune alerte haute ou critique
Configurations Compose	Trois configurations valides
Build Docker standalone	Succès sous Node.js 22, exécution avec l'utilisateur <code>nextjs</code>
Healthcheck avec MariaDB migrée	HTTP 200, statut <code>ok</code>
Lighthouse	Seuils réussis sur accueil, connexion et inscription

Preuve GitHub Actions

L'exécution GitHub Actions no 29733455501 a été réalisée sur le SHA `25d0de5a3ee5167e6cba5299dce0196a4bb0d188` le 20 juillet 2026.

Élément distant	Résultat
Job <code>Quality gates</code>	Succès

Élément distant	Résultat
Job Lighthouse thresholds	Succès
Artefact de couverture	coverage - 25d0de5a3ee5167e6cba5299dce0196a4bb0d188 , 38 041 octets
Artefact Lighthouse	lighthouse - 25d0de5a3ee5167e6cba5299dce0196a4bb0d188 , 328 684 octets
Durée de conservation	30 jours

Cette exécution constitue la preuve reproductible initiale de C2.1.2. Une capture du résumé des deux jobs sera intégrée au dossier final afin que la preuve reste lisible après expiration des artefacts.

Validation du lot prototype C2.2.1

L'exécution [GitHub Actions no 29739778393](#) a validé le SHA `1db84b8e1058aea6c4785fa5b14a9fdcde533541` le 20 juillet 2026 après l'ajout des parcours matching et événements.

Élément distant	Résultat
Job Quality gates	Succès : lint, TypeScript, 69 tests, couverture, audit, configurations Docker et build
Job Lighthouse thresholds	Succès
Statut global	Succès

Cette seconde exécution prouve que le prototype décrit dans le document C2.2.1 est reproductible sur l'environnement d'intégration, et pas uniquement sur le poste de développement.

Validation des tests d'intégration C2.2.2

L'exécution [GitHub Actions no 29740751753](#) a validé le SHA `1e1cb16a9908257dba9b2e8f8e251c672ac0a6d4` le 20 juillet 2026.

Job	Résultat
Quality gates	Succès
MariaDB integration tests	Succès : service sain, migrations complètes et 10 résultats TAP
Lighthouse thresholds	Succès
Artefact d'intégration	integration - 1e1cb16a9908257dba9b2e8f8e251c672ac0a6d4 , 898 octets, conservé 30 jours

Cette exécution est la preuve distante que les migrations et les parcours critiques fonctionnent ensemble sur une base créée à neuf.

Validation sécurité et accessibilité C2.2.3

L'exécution [GitHub Actions no 29743712530](#) a validé le SHA `3779eee23e86f013c76e54f8f96a44a87a80b31c` le 20 juillet 2026.

Job	Résultat
Quality gates	Succès : 83 tests, couverture, audit de dépendances, lint, TypeScript et build

Job	Résultat
MariaDB integration tests	Succès : 11 résultats TAP puis audit authentifié sur dix états
Lighthouse thresholds	Succès : accessibilité 100 % sur les trois pages publiques
Artefact de couverture	coverage-3779eee23e86f013c76e54f8f96a44a87a80b31c , 53 385 octets
Artefact d'intégration	integration-3779eee23e86f013c76e54f8f96a44a87a80b31c , 961 octets
Artefact Lighthouse public	lighthouse-3779eee23e86f013c76e54f8f96a44a87a80b31c , 327 941 octets
Artefact accessibilité authentifiée	accessibility-3779eee23e86f013c76e54f8f96a44a87a80b31c , 1 509 342 octets
Expiration des artefacts	19 août 2026

Cette exécution prouve sur une base et un runner neufs le quota HTTP, les corrections RGAA, les scores Lighthouse et la génération des captures mobiles décrites dans les documents C2.2.3.

Validation du runtime GitHub Actions Node.js 24

L'exécution [GitHub Actions no 29744313577](#) a validé le SHA `8be8a29445d9eaa07b6e703c45b0b4622f5464e4` le 20 juillet 2026 après la migration vers `actions/setup-node@v6` et `actions/upload-artifact@v6`.

Contrôle	Résultat
Quality gates	Succès, aucune annotation
MariaDB integration tests	Succès, aucune annotation
Lighthouse thresholds	Succès, aucune annotation
Runtime des actions JavaScript concernées	Node.js 24 natif

Cette exécution confirme la disparition de l'avertissement de dépréciation Node.js 20 sans régression sur les contrôles ni sur la publication des quatre artefacts.

8. Limites et évolutions prévues

- La CI prouve actuellement la qualité du périmètre unitaire ciblé, pas encore la majorité de l'application complète.
- Les composants critiques et l'échantillon RGAA sont pilotés par axe, Lighthouse et Puppeteer ; le cahier de recette devra encore couvrir l'ensemble des parcours visuels nominaux et alternatifs.
- Les contrôles d'accessibilité restent séparés des tests métier afin de conserver des diagnostics lisibles.
- Le déploiement continu restera séparé de la CI : une version ne sera promue qu'après le succès des contrôles et une validation explicite de l'environnement cible.

Harnais de tests unitaires - C2.2.2

1. Objectif et périmètre initial

Le harnais de tests prévient les régressions sur les règles métier déjà isolées de Spity. Il couvre actuellement :

- la validation des comptes, mots de passe, profils, cotations, contenus et événements ;
- la validation et la normalisation du profil public et du matériel ;
- l'analyse d'une saisie libre de matériel d'escalade ;
- le contrôle d'origine utilisé contre les requêtes intersites ;
- les règles de filtrage, de paire unique et de conversion des données de matching ;
- les règles de dates, de capacité et d'inscription aux événements ;
- les états clavier et chargement du composant `Button`.

Cette première couverture est volontairement mesurée sur les fichiers déclarés dans `jest.config.ts`. Elle ne doit pas être interprétée comme une couverture globale de toute l'application.

2. Outils et configuration

Outil	Rôle
Jest	Exécution et assertions des tests unitaires
<code>next/jest</code>	Transformation TypeScript/JSX cohérente avec Next.js
jsdom	Simulation de l'environnement du navigateur
React Testing Library	Test des composants selon leur comportement visible
<code>@testing-library/user-event</code>	Simulation des interactions clavier et utilisateur
V8 coverage	Mesure des lignes, instructions, fonctions et branches

Commandes disponibles :

```
npm test
npm run test:watch
npm run test:coverage
```

Les seuils bloquants du périmètre mesuré sont :

- 80 % pour les lignes, instructions et fonctions ;
- 70 % pour les branches ;
- 100 % des tests réussis.

3. Organisation des tests

Fichier de test	Risques contrôlés
<code>src/lib/validators.test.ts</code>	Entrées invalides, mot de passe faible, cotation incorrecte, contenu vide, capacité invalide
<code>src/features/profile/schemas.test.ts</code>	Normalisation des champs, coercition des formulaires, limites de quantité et de disponibilité
<code>src/features/profile/lib/equipment-parser.test.ts</code>	Catégorie, marque, modèle, quantité, dimensions, séparateurs et valeurs inconnues
<code>src/features/auth/lib/csrf.test.ts</code>	Origine absente, identique, externe, invalide et réponse 403 typée
<code>src/features/matching/lib/matching-rules.test.ts</code>	Clé de paire stable, JSON MariaDB, filtres combinés, localité, niveau et disponibilité
<code>src/features/events/lib/event-rules.test.ts</code>	Dates futures, ordre début/fin, capacité et refus des inscriptions invalides
<code>src/features/events/schemas.test.ts</code>	Contrats de création, modification, annulation et formulaire d'événement
<code>src/components/ui/button.test.tsx</code>	Activation au clavier, focus, verrouillage et état <code>aria-busy</code>

Les tests suivent le modèle AAA : préparation des données, exécution du comportement, puis assertions sur le résultat observable.

4. Résultat de référence

Exécution locale du 20 juillet 2026 :

```
Test Suites: 8 passed, 8 total
Tests:      69 passed, 69 total
```

Indicateur	Résultat	Seuil	Statut
Instructions	97,87 %	80 %	Conforme
Lignes	97,87 %	80 %	Conforme
Fonctions	100 %	80 %	Conforme
Branches	90,90 %	70 %	Conforme

Le rapport HTML est généré dans `spity/coverage/lcov-report/`. Ce dossier est ignoré par Git ; la CI conserve le rapport comme artefact pendant 30 jours.

5. Complément d'intégration MariaDB

Les 69 tests Jest restent des tests unitaires rapides. Ils sont complétés par le scénario HTTP décrit dans [06 TESTS INTEGRATION C222.md](#), qui applique les migrations sur MariaDB et traverse les Route Handlers, la session, les dépôts Drizzle et les contraintes de base.

Le résultat local de référence est de 10 contrôles TAP réussis, dont deux inscriptions concurrentes sur la dernière place d'un événement.

6. Régression détectée par le harnais

Identifiant provisoire : `BUG-TEST-001`.

Champ	Valeur
Contexte	Analyse de la saisie libre d'un inventaire de matériel
Entrée	Beal Joker corde 60 m 9,1 mm turquoise
Résultat incorrect	La virgule décimale séparait l'entrée en deux articles ; le diamètre et la couleur étaient perdus
Cause	L'expression de découpage traitait toutes les virgules comme des séparateurs de liste
Correction	Ne séparer une virgule que lorsqu'elle n'introduit pas la partie décimale d'un nombre
Défaut associé	Une catégorie inconnue produisait aussi un modèle générique au lieu de conserver le libellé saisi
Retest	Les 49 tests passent et le parseur atteint 100 % sur les quatre indicateurs

Cet exemple alimentera le plan de correction C2.3.2 avec le SHA du commit et la preuve avant/après.

7. Limites de la couverture actuelle

La grille demande que les tests couvrent la majorité du code développé. Ce résultat n'est pas encore atteint à l'échelle des 8 000 lignes du projet.

Les prochains lots devront couvrir :

1. la création et la vérification des sessions ;
2. le verrouillage après échecs de connexion ;
3. les dépôts de matériel avec une base de test ;
4. les routes du matériel et les formulaires non inclus dans le prototype BC02 ;
5. les formulaires d'authentification et de profil ;
6. la mesure de couverture de lignes des dépôts matching, partenariats et événements ;
7. les parcours visuels avec Playwright, en complément des tests HTTP.

Les Server Components asynchrones seront vérifiés par des tests de bout en bout, conformément à la recommandation actuelle de Next.js.

Architecture et prototype maintenable - C2.2.1

1. Objet et périmètre démontré

Ce document présente l'architecture réellement livrée pour la compétence C2.2.1 du bloc BC02. Le prototype évalué couvre deux parcours métier complets en plus de l'authentification et des profils déjà présents :

- recherche de grimpeurs, filtrage et gestion des demandes de partenariat ;
- publication d'événements par un club, consultation, inscription et suivi des participants.

Les user stories de référence sont décrites dans [01_PERIMETRE_FONCTIONNEL_ET_USER_STORIES.md](#) . Le fil social, la messagerie, les topos collaboratifs et le calcul de distance GPS restent hors du prototype BC02.

2. Choix techniques vérifiés

Élément	Choix dans la version livrée	Justification
Framework web	Next.js 16.2.10, App Router	Routage, rendu serveur, composants client et API dans un même projet TypeScript.
Interface	React 19, Tailwind CSS, composants <code>src/components/ui/</code>	Réutilisation du design system et adaptation responsive sans dupliquer les contrôles.
Formulaires	React Hook Form et Zod	Validation accessible côté client, puis nouvelle validation Zod côté API.
Backend	Route Handlers Next.js	Endpoints colocalisés avec l'application et contrôles d'accès exécutés sur le serveur.
Persistance	MariaDB 11.4 et Drizzle ORM	Modèle typé, migrations versionnées et requêtes sans SQL brut dans le code applicatif.
Tests	Jest 30	Règles métier pures testées sans dépendre de l'interface ou de MariaDB.
Livraison	Docker et GitHub Actions	Environnements reproductibles et contrôles automatisés avant intégration.

Le dépôt impose TypeScript strict. Les structures externes sont traitées comme `unknown` , validées, puis converties en types métier. Les réponses des nouveaux endpoints sont elles aussi vérifiées par des schémas Zod avant envoi.

3. Vue en couches

Le schéma source est versionné dans [architecture-prototype-c221.mmd](#) .

```

flowchart TB
    U[Grimpeur ou club]
    UI[Pages App Router et composants React]
    API[Route Handlers /api]
    AUTH[Session, rôle et contrôle d'origine]
    ZOD[Contrats Zod entrée et sortie]
    RULES[Règles métier pures]
    REPO[Dépôts Drizzle]
    DB[(MariaDB)]
    CI[GitHub Actions]

    U --> UI
    UI --> API
    API --> AUTH
    API --> ZOD
    API --> RULES
    API --> REPO
    REPO --> DB
    RULES -. testées par Jest .-> CI
    ZOD -. validé par lint, typecheck et tests .-> CI

```

Présentation

Les pages serveur chargent l'utilisateur et les données initiales, puis redirigent un visiteur non authentifié. Les composants client gèrent les filtres, formulaires et retours immédiats :

- `src/app/app/matching/page.tsx` et `src/features/matching/components/matching-directory.tsx` ;
- `src/app/app/partnerships/page.tsx` et `src/features/matching/components/partnership-center.tsx` ;
- `src/app/app/events/page.tsx`, `events-board.tsx` et `event-form.tsx`.

Les contrôles possèdent des libellés, les messages dynamiques utilisent `aria-live` ou `role="alert"`, et les commandes restent utilisables au clavier. Les grilles passent de une à plusieurs colonnes selon les points de rupture Tailwind.

API et application

Route	Méthode	Rôle	Traitement
<code>/api/matching</code>	GET	Grimpeur	Retourne au maximum 100 profils publics ouverts au matching, sans adresse électronique.
<code>/api/partnerships</code>	GET , POST	Grimpeur	Liste ou crée une demande unique pour une paire.
<code>/api/partnerships/[requestId]</code>	PATCH	Destinataire	Accepte ou refuse uniquement une demande en attente.
<code>/api/events</code>	GET , POST	Authentifié / club	Liste l'agenda ou crée un événement futur.
<code>/api/events/[eventId]</code>	PATCH	Club propriétaire	Modifie ou annule un événement du club.
<code>/api/events/[eventId]/registrations</code>	POST , DELETE	Grimpeur	Inscrit, désinscrit ou réactive une inscription.

Chaque mutation suit la séquence : contrôle d'origine, session, rôle, paramètres, corps Zod, autorisation sur la ressource, règle métier, écriture Drizzle et réponse Zod.

Métier

Les règles indépendantes de l'infrastructure sont regroupées dans :

- `matching-rules.ts` : conversion sûre des JSON, clé de paire stable et combinaison des filtres ;
- `event-rules.ts` : cohérence des dates, capacité minimale et conditions d'inscription.

Ce découpage permet de tester rapidement les cas nominaux et les limites. L'annuaire est ordonné par nom d'affichage puis identifiant, ce qui rend les résultats déterministes. Les filtres localité/nom, discipline, niveau, disponibilité et environnement sont combinés.

Données

La migration `drizzle/0004_majestic_ken_ellis.sql` ajoute :

- l'unicité d'un profil par utilisateur ;
- `partnership_requests` , avec une clé de paire unique et les statuts `pending` , `accepted` , `declined` ;
- les informations de type, description, lieu, fin et statut des événements ;
- `event_registrations` , unique par couple événement/utilisateur.

Une inscription est réalisée dans une transaction qui verrouille la ligne de l'événement avec `FOR UPDATE` . La capacité est recalculée sous verrou avant insertion ou réactivation. Deux requêtes concurrentes ne peuvent donc pas valider la même dernière place.

4. Paradigmes et maintenabilité

- **Architecture par fonctionnalité** : matching et événements possèdent chacun schémas, composants, règles, réponses et dépôt.
- **Rendu hybride** : les Server Components assurent le chargement initial et l'autorisation ; les Client Components ne prennent en charge que l'interaction.
- **Programmation fonctionnelle** : les validations de dates, capacité et filtres sont des fonctions pures sans état global.
- **Repository** : Drizzle et la forme des tables restent confinés aux fichiers d'accès aux données.
- **Défense en profondeur** : validation client pour l'ergonomie, validation serveur pour la sécurité, contraintes uniques en base pour l'intégrité.
- **Évolution additive** : une nouvelle migration est générée ; aucune migration historique n'est modifiée.

Ajouter un nouveau type d'événement demande une évolution coordonnée de l'enum Drizzle, du schéma Zod et des libellés d'interface. Ajouter un nouveau filtre de matching reste localisé au contrat, à la fonction pure et au contrôle d'interface.

5. Traçabilité des user stories

User story	Interface	Règle ou protection	Preuve exécutée le 20 juillet 2026
US-MATCH-01	<code>/app/matching</code>	Profil courant exclu, profil public uniquement, filtres combinés et ordre stable	Page HTTP 200, API retournant uniquement Nassim pour le compte Lina.
US-MATCH-02	<code>/app/matching</code> , <code>/app/partnerships</code>	Paire unique, auto-demande refusée, réponse réservée au destinataire	Demande Lina vers Nassim puis acceptation avec le compte Nassim.
US-EVENT-01	<code>/app/events</code> , formulaire club	Dates futures, fin après début, propriétaire requis, capacité protégée	Création, modification de capacité et annulation par le compte club.

User story	Interface	Règle ou protection	Preuve exécutée le 20 juillet 2026
US-EVENT-02	/app/events	Événements futurs triés, capacité calculée	Deux événements de démonstration retournés dans l'ordre chronologique.
US-EVENT-03	/app/events	Transaction, verrou, unicité et réactivation	Désinscription puis réinscription de Lina, compteur passé de 1 à 0 puis à 1.
US-EVENT-04	/app/events	Participants fournis uniquement au club propriétaire	Réponse club avec <code>isOwner: true</code> ; réponses grimpeur avec <code>participants: []</code> .

6. Vérifications réalisées

Contrôle	Résultat
Migration complète sur MariaDB temporaire	Succès, migrations 0000 à 0004 appliquées.
Seed reproductible	Succès, 2 événements, 1 demande et 2 inscriptions actives.
Routes des trois écrans connectés	HTTP 200.
Tests unitaires	69 tests réussis sur 69.
ESLint	Succès, aucune erreur ni avertissement.
TypeScript strict	Succès.
GitHub Actions	Exécution 29739778393 réussie sur le SHA 1db84b8e1058aea6c4785fa5b14a9fdcde533541.

Les comptes du seed permettent une démonstration immédiate avec le mot de passe documenté dans le script : Lina envoie les demandes, Nassim possède une demande entrante, et Club Alpin Lyon administre les événements.

7. Limites et suites

- La recherche par localité est textuelle ; le rayon géographique et la cartographie sont reportés.
- La limite de 100 profils convient au prototype. Une pagination et des index de recherche seront nécessaires à plus grande échelle.
- Les contrôles HTTP et la concurrence d'inscription sont automatisés avec MariaDB dans le job d'intégration C2.2.2.
- Les captures desktop et mobile seront ajoutées à l'annexe visuelle du dossier final.
- Un audit RGAA complet et la matrice OWASP restent traités dans C2.2.3 ; ce document ne revendique pas une conformité globale.

Ces limites sont explicites afin de distinguer une architecture démontrable et maintenable d'une version de production à grande échelle.

Tests d'intégration du prototype - C2.2.2

1. Objectif

Ce document complète le harnais unitaire décrit dans [04_HARNAIS_TESTS_UNITAIRES.md](#). Il démontre les parcours critiques du prototype BC02 avec les composants réels suivants :

- serveur Next.js et Route Handlers ;
- cookies de session signés ;
- validation Zod et autorisations par rôle ;
- dépôts Drizzle ;
- migrations 0000 à 0004 ;
- base MariaDB 11.4.

Le test source est [spity/tests/integration/api-workflows.test.mjs](#). Il utilise le module `node:test` fourni par Node.js 22 et l'API native `fetch`, sans mock du réseau ni de la base.

2. Stratégie retenue

Le scénario crée trois comptes éphémères avec un suffixe UUID : deux grimpeurs et un club. Il les utilise exclusivement via les endpoints HTTP publics. Les identifiants sont supprimés dans le hook de fin, même en cas d'échec, et les suppressions en cascade nettoient profils, demandes, événements et inscriptions.

Lorsqu'aucune URL n'est fournie, le test :

1. démarre Next.js sur `127.0.0.1:3102` ;
2. attend que `/api/health` confirme l'accès à MariaDB ;
3. exécute les scénarios dans un ordre déterministe ;
4. nettoie les comptes de test ;
5. arrête le processus Next.js.

Pour tester un serveur déjà actif, la variable `INTEGRATION_BASE_URL` évite de créer un second processus :

```
INTEGRATION_BASE_URL=http://127.0.0.1:3000 npm run test:integration
```

3. Cas automatisés

ID	Domaine	Action	Résultat contrôlé
IT-AUTH-01	Accès	Appeler le matching sans cookie	Réponse <code>401</code> , aucune donnée retournée.
IT-PROFILE-01	Profils	Créer deux grimpeurs et un club	Réponses <code>201</code> , profils correspondant aux rôles.
IT-MATCH-01	Matching	Consulter l'annuaire avec le premier grimpeur	Profil courant absent, second profil présent, champ <code>email</code> absent.
IT-MATCH-02	Demandes	S'envoyer une demande	Réponse <code>422</code> .
IT-MATCH-03	Demandes	Créer deux fois la même demande	Première réponse <code>201</code> , seconde <code>409</code> .

ID	Domaine	Action	Résultat contrôlé
IT-MATCH-04	Autorisation	Répondre comme émetteur puis comme destinataire	Émetteur refusé 403 , destinataire accepté 200 .
IT-SEC-01	Origine	Muter avec une origine externe	Réponse 403 .
IT-EVENT-01	Rôle	Créer un événement avec un grimpeur	Réponse 403 .
IT-EVENT-02	Club	Créer un événement futur d'une place	Réponse 201 , capacité restante égale à 1.
IT-EVENT-03	Concurrence	Envoyer deux inscriptions en parallèle	Une réponse 200 , une réponse 409 , un seul participant en base.
IT-EVENT-04	Confidentialité	Lire l'événement comme club puis grimpeur	Le club voit un participant ; le grimpeur reçoit participants: [] .
IT-EVENT-05	Désinscription	Libérer puis reprendre la dernière place	Compteur remis à zéro puis nouvelle inscription 200 .
IT-EVENT-06	Annulation	Annuler puis tenter une nouvelle inscription	Événement traçable comme annulé, inscription refusée 409 .

Le cas IT-EVENT-03 sollicite directement la transaction et le verrou `FOR UPDATE` du dépôt. Il prouve que deux requêtes concurrentes ne dépassent pas la capacité, ce qu'un test unitaire isolé ne peut pas démontrer.

4. Intégration continue

Le workflow [.github/workflows/ci.yml](#) contient un job indépendant `MariaDB integration tests` :

1. GitHub Actions démarre le service `mariadb:11.4` avec healthcheck ;
2. `npm ci` installe les versions verrouillées ;
3. `npm run db:migrate` applique tout l'historique Drizzle sur une base vide ;
4. `npm run test:integration` démarre Next.js et exécute le scénario ;
5. le résultat TAP est conservé dans l'artefact `integration-<SHA>` pendant 30 jours.

Le job est séparé de `Quality gates`. Un échec indique ainsi clairement si la régression concerne les tests unitaires/compilation ou l'intégration avec MariaDB.

5. Résultat local de référence

Exécution du 20 juillet 2026 sur Node.js 22, Next.js 16.2.10 et MariaDB 11.4 :

```
tests 10
pass 10
fail 0
duration_ms 14467.552506
```

Les dix résultats TAP correspondent au scénario parent et à neuf sous-scénarios. Les 69 tests Jest continuent de s'exécuter séparément avec 97,87 % de lignes, 90,90 % de branches et 100 % de fonctions sur le périmètre unitaire déclaré.

Après l'exécution sur la base initialement vide, les tables `users`, `events`, `partnership_requests` et `event_registrations` contenaient chacune zéro ligne. Ce contrôle confirme le nettoyage des données éphémères.

Résultat GitHub Actions

L'exécution no 29740751753 a réussi sur le SHA 1e1cb16a9908257dba9b2e8f8e251c672ac0a6d4 :

- job MariaDB integration tests réussi ;
- migrations appliquées sur le service mariadb:11.4 neuf ;
- 10 résultats TAP réussis ;
- artefact integration-1e1cb16a9908257dba9b2e8f8e251c672ac0a6d4 de 898 octets conservé jusqu'au 19 août 2026.

6. Traçabilité et couverture

User story	Preuve d'intégration
US-AUTH-01 / US-AUTH-02	Création de comptes, cookies de session et refus anonyme.
US-PROFILE-01 / US-PROFILE-02	Création et lecture des profils correspondant aux rôles.
US-MATCH-01	Annuaire réel, exclusion du profil courant et confidentialité de l'adresse.
US-MATCH-02	Unicité, auto-demande, autorisation du destinataire et acceptation.
US-EVENT-01	Création par le club, contrôle du rôle et annulation persistée.
US-EVENT-02	Lecture de l'événement, capacité et données publiques.
US-EVENT-03	Concurrence, unicité, désinscription et réactivation.
US-EVENT-04	Compteurs réels et participants visibles uniquement par le propriétaire.

7. Limites

- Le test exerce les dépôts au travers des API, sans mesurer séparément leur couverture de lignes.
- Les formulaires React et le rendu visuel ne sont pas pilotés dans un navigateur ; Playwright restera nécessaire pour la recette d'interface.
- Le scénario vérifie une seule instance Next.js et MariaDB. Un test de charge distribué dépasse le périmètre du prototype.
- La preuve distante dépend d'un artefact conservé 30 jours ; une capture du résumé du job sera ajoutée à l'annexe finale.

Ces limites n'empêchent pas la validation des règles critiques de C2.2.2 : elles identifient les contrôles complémentaires à traiter dans le cahier de recette C2.3.1.

Mesures de sécurité et matrice OWASP Top 10 - C2.2.3

1. Objectif et référence

Ce document présente les mesures de sécurité intégrées au prototype Spity pour la compétence C2.2.3. La référence retenue est l'[OWASP Top 10:2025](#), version publiée la plus récente au 20 juillet 2026.

L'OWASP Top 10 est un document de sensibilisation aux risques, pas une certification de conformité. La méthode appliquée à Spity consiste donc à relier chaque catégorie à :

- une menace concrète du prototype ;
- une ou plusieurs mesures présentes dans le code ou l'infrastructure ;
- une preuve automatisée ou reproductible ;
- un risque résiduel assumé et une suite identifiée.

2. Périmètre audité

L'audit porte sur les parcours BC02 suivants :

- inscription, connexion, session et déconnexion ;
- création et modification des profils grimpeur et club ;
- annuaire de matching et demandes de partenariat ;
- création, modification et annulation d'événements ;
- inscription concurrente et désinscription à un événement ;
- chaîne npm, GitHub Actions, image Docker et configuration HTTP.

Les fonctions hors périmètre du prototype, par exemple paiement, upload média et messagerie temps réel, ne sont pas présentées comme sécurisées ou auditées.

3. Corrections issues de l'audit

ID	Écart constaté	Correction mise en œuvre	Preuve
SE C-01	Les en-têtes ne comportaient pas de CSP.	CSP centralisée, interdiction des objets et frames, restriction des sources, HSTS en production, politiques cross-origin et suppression de <code>X-Powered-By</code> .	<code>security-headers.ts</code> , tests unitaires et réponse HTTP de production.
SE C-02	L'ancien <code>middleware.ts</code> de limitation n'était plus exécuté avec la convention Next.js 16.	Contrôle déplacé dans les Route Handlers d'authentification, au plus près de la ressource sensible.	Dix tentatives en <code>401</code> , onzième en <code>429</code> avec <code>Retry-After</code> dans le test HTTP/MariaDB.
SE C-03	Le quota d'authentification était identique au quota API générique.	Fenêtre dédiée de 10 tentatives par client sur 15 minutes, en plus du verrouillage temporaire du compte après cinq échecs.	<code>rate-limit.test.ts</code> et cas d'intégration IT-SEC-02.
SE C-04	Le contrôle CSRF reposait uniquement sur <code>Origin</code> .	Rejet supplémentaire des requêtes marquées <code>Sec-Fetch-Site: cross-site</code> .	<code>csrf.test.ts</code> et cas IT-SEC-01.

ID	Écart constaté	Correction mise en œuvre	Preuve
SE C-05	Les événements de sécurité n'étaient pas structurés.	Logger JSON centralisé pour succès/échec de connexion, verrouillage, quota, inscription et origine rejetée, sans email, mot de passe ni jeton.	<code>logger.ts</code> et sorties serveur structurées.
SE C-06	Aucun repli utilisateur dédié n'existait pour une erreur de rendu.	Error boundary avec message neutre, relance et retour à l'accueil.	<code>src/app/error.tsx</code> , build Next.js réussi.
SE C-07	La surveillance des mises à jour était uniquement manuelle.	Configuration Dependabot hebdomadaire pour npm et GitHub Actions, ciblant <code>develop</code> .	<code>.github/dependabot.yml</code> .

La migration de `middleware.ts` vers `proxy.ts` est documentée par [Next.js 16](#). Le rate limiting ne repose pas sur Proxy, car la [documentation Proxy](#) déconseille explicitement de dépendre d'un état partagé à cette frontière réseau.

4. Matrice OWASP Top 10:2025

Catégorie	Risque appliqué à Spity	Mesures et preuves	État
A01 Broken Access Control	Lecture d'un profil privé, modification de l'événement d'un autre club, traitement d'une demande par l'émetteur, CSRF.	Session contrôlée côté serveur ; rôles <code>grimpeur / club</code> ; vérification du propriétaire ou destinataire ; réponses <code>401 / 403</code> ; cookie <code>SameSite=Lax</code> ; contrôle <code>Origin</code> et Fetch Metadata. Les tests refusent l'anonyme, l'émetteur non autorisé, le mauvais rôle et masquent les participants aux non-propriétaires.	Couvert
A02 Security Misconfiguration	Configuration par défaut trop permissive, fuite de technologie, clicjacking, chargement de ressources externes.	Variables validées par Zod ; secrets obligatoires ; CSP ; <code>frame-ancestors 'none'</code> ; <code>X-Frame-Options: DENY</code> ; HSTS en production ; <code>nosniff</code> ; Permissions Policy ; politiques cross-origin ; base de production non exposée ; <code>poweredByHeader: false</code> .	Couvert avec réserve CSP
A03 Software Supply Chain Failures	Dépendance compromise ou vulnérable, action CI obsolète, installation non reproductible.	<code>package-lock.json</code> , <code>npm ci</code> , audit de production bloquant sur sévérité haute, Dependabot npm/Actions, build et tests dans une CI en lecture seule.	Couvert avec dette modérée suivie
A04 Cryptographic Failures	Mot de passe lisible, secret faible, cookie intercepté ou jeton falsifié.	bcrypt avec facteur 12 et entrée bornée à 72 octets ; secret HMAC d'au moins 32 caractères ; comparaison de signature en temps constant ; expiration de sept jours ; cookie <code>HttpOnly</code> , <code>Secure</code> en production et <code>SameSite=Lax</code> ; HSTS.	Couvert pour le prototype
A05 Injection	Injection SQL, script utilisateur, paramètres ou JSON malformés.	Entrées validées et bornées par Zod ; paramètres dynamiques contrôlés ; Drizzle et requêtes paramétrées ; rendu React échappé ; absence de <code>dangerouslySetInnerHTML</code> , <code>eval</code> métier et SQL concaténé ; CSP en défense complémentaire.	Couvert
A06 Insecure Design	Contournement d'une transition métier, doublon ou dépassement de capacité.	User stories avec cas négatifs ; refus de l'auto-demande ; paire de partenariat unique ; statuts bornés ; transaction avec verrou <code>FOR UPDATE</code> sur la dernière place ; contraintes de base ; quota borné en mémoire.	Couvert pour le périmètre
A07 Authentication Failures	Brute force, énumération, session faible ou compte forcé.	Erreur de connexion générique ; comparaison bcrypt factice lorsqu'un compte est absent pour limiter l'écart temporel ; politique de mot de passe ; verrouillage temporaire après cinq échecs ; quota client 10/15 min ; session signée et expirante ; journalisation des échecs. Le test d'intégration observe réellement le <code>429</code> .	Couvert avec limite distribuée

Catégorie	Risque appliqué à Spity	Mesures et preuves	État
A08 Software or Data Integrity Failures	Donnée ou artefact altéré, réponse incohérente, migration non maîtrisée.	Migrations additives versionnées ; lockfile ; réponses API parsées par Zod ; contraintes et transactions MariaDB ; branches <code>develop / main</code> contrôlées par CI ; image construite depuis les sources verrouillées.	Couvert avec réserve sur la signature des releases
A09 Security Logging and Alerting Failures	Échec d'authentification ou origine hostile invisible ; fuite de secret dans les logs.	Événements JSON horodatés et nommés ; identifiant interne seulement lorsqu'il existe ; aucun mot de passe, email ou jeton ; artefacts et résultats CI conservés 30 jours.	Partiel : collecte présente, alerting centralisé absent
A10 Mishandling of Exceptional Conditions	JSON invalide, erreur de base, état concurrent ou erreur de rendu provoquant une fuite ou une incohérence.	Lecture JSON protégée ; réponses typées <code>4xx / 5xx</code> sans stack ; healthcheck neutre ; transactions ; error boundary accessible ; arrêt bloquant des scripts en cas d'échec.	Couvert pour les cas testés

La liste et les intitulés sont ceux de l'[introduction officielle OWASP Top 10:2025](#).

5. En-têtes HTTP vérifiés

Après un build de production, `HEAD /login` retourne notamment :

```
Content-Security-Policy: default-src 'self'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'; object-src 'none'; ...
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Permissions-Policy: camera=(), microphone=(), geolocation=(), payment=(), usb=()
Referrer-Policy: strict-origin-when-cross-origin
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
```

`X-Powered-By` est désactivé. HSTS n'est volontairement ajouté qu'en production. En développement, le CSP autorise `unsafe-eval` et les WebSockets nécessaires au rechargement Next.js ; ces permissions ne sont pas présentes dans la politique de production.

La politique statique suit l'option sans nonce décrite dans le [guide CSP officiel de Next.js](#). Elle conserve `unsafe-inline` pour les scripts et styles injectés par le rendu statique Next.js : cette limite est suivie dans les risques résiduels.

6. Résultats reproductibles

Commandes exécutées le 20 juillet 2026 :

```
npm run lint
npm run typecheck
npm run test:coverage
npm run test:integration
npm run security:audit
npm run build
```

Résultats :

- lint et TypeScript strict sans erreur ;

- 83 tests Jest réussis ; couverture de 96 % des lignes et instructions, 89,69 % des branches et 100 % des fonctions sur le périmètre déclaré ;
- 11 résultats TAP réussis, dont le contrôle de quota HTTP et la concurrence MariaDB ;
- aucune vulnérabilité de production critique ou haute signalée par `npm audit` ;
- build de production réussi ;
- en-têtes contrôlés sur le serveur standalone.

Le scénario de quota utilise une adresse réservée à la documentation, envoie onze connexions invalides avec la même identité réseau et vérifie :

```
tentatives 1 à 10 : 401 Unauthorized
tentative 11      : 429 Too Many Requests
X-RateLimit-Remaining: 0
Retry-After: valeur strictement positive
```

La preuve distante est l'[exécution GitHub Actions no 29743712530](#), réussie sur le SHA `3779eee23e86f013c76e54f8f96a44a87a80b31c`. Les jobs `Quality gates`, `MariaDB integration tests` et `Lighthouse thresholds` sont tous réussis ; les rapports sont conservés jusqu'au 19 août 2026.

7. Risques résiduels et décisions

Risque résiduel	Niveau	Décision pour le prototype	Action avant production multi-instance
Deux alertes npm modérées concernent le PostCSS embarqué par Next.js.	Modéré	Accepté temporairement : aucune CSS n'est générée depuis une entrée utilisateur ; <code>npm audit fix --force</code> propose un downgrade majeur incohérent.	Mettre à jour Next.js dès la publication d'une version embarquant PostCSS corrigé et conserver la CI bloquante sur haut/critique.
Le CSP statique contient <code>unsafe-inline</code> .	Modéré	Accepté avec React échappé, absence d'HTML injecté et sources externes fortement restreintes.	Étudier des nonces avec rendu dynamique ou SRI lorsque l'option Turbopack sera stable, puis mesurer le coût de performance.
Le quota est stocké par processus.	Modéré	Suffisant pour la démonstration mono-instance et explicitement testé.	Utiliser Redis avec clé IP anonymisée, fenêtre atomique et quota partagé entre instances.
Un jeton signé reste valide jusqu'à expiration après compromission.	Modéré	Déconnexion côté navigateur et durée limitée à sept jours.	Ajouter identifiant de session, stockage serveur, révocation et rotation du secret.
L'inscription révèle qu'une adresse existe déjà.	Faible à modéré	Message utile pour le MVP ; la connexion reste générique.	Réponse neutre ou workflow de récupération de compte par email.
Les logs ne déclenchent pas d'alerte.	Modéré	Traces structurées disponibles dans le runtime et la CI.	Centraliser, définir une rétention, masquer les identifiants et alerter sur <code>rate_limit_exceeded</code> , verrouillages et pics de 403.
Les actions GitHub utilisent des versions majeures, pas des SHA immuables.	Faible	Dependabot surveille leur mise à jour.	Épingler les actions à des SHA vérifiés dans le durcissement de production.

L'alerte PostCSS est suivie par l'[avis GitHub GHSA-qx2v-qp2m-jg93](#). Elle n'est ni masquée ni présentée comme corrigée.

8. Conclusion

Les dix catégories OWASP 2025 sont couvertes par au moins une mesure concrète et une preuve sur le périmètre BC02. A09 reste partiel faute de centralisation et d'alerting, et trois limites de production sont explicitement reportées : CSP stricte avec nonce, quota distribué et révocation serveur des sessions.

Cette conclusion vaut pour le prototype audité. Elle ne constitue pas un test d'intrusion ni une garantie absolue contre toute vulnérabilité.

Accessibilité du prototype et audit RGAA 4.1.2 - C2.2.3

1. Référentiel choisi

Spity retient le [Référentiel général d'amélioration de l'accessibilité 4.1.2](#) comme référence de C2.2.3.

Ce choix est justifié par quatre éléments :

- 1. il s'agit du référentiel technique officiel français applicable aux services web au moment de l'audit ;
- 2. ses 106 critères fournissent une méthode de contrôle testable, organisée en 13 thèmes ;
- 3. il est cohérent avec le contexte français du titre RNCP et du public de Spity ;
- 4. il s'appuie sur WCAG 2.1, dont les critères sont organisés autour des contenus perceptibles, utilisables, compréhensibles et robustes.

La DINUM indique que le RGAA 5 est en cours de rédaction pour une publication prévue fin 2026 et que les travaux fondés sur la version 4.1.2 ne doivent pas être reportés. La référence 4.1.2 reste donc adaptée au dossier remis en juillet 2026. Les règles de conformité WCAG imposent par ailleurs de considérer les pages complètes et les processus complets, pas seulement un composant isolé ([WCAG 2.1, section conformité](#)).

2. Portée de l'audit

Échantillon public

État	Route	Parcours
Accueil	/	Comprendre le service et accéder à l'authentification.
Connexion	/login	Renseigner les identifiants, afficher le mot de passe, recevoir une erreur.
Inscription	/register	Créer un compte et choisir un rôle.

Échantillon authentifié

Rôle	Route ou état	Parcours
Grimpeur	/app	Tableau de bord et navigation principale.
Grimpeur	/app/matching	Recherche, filtres et fiches de partenaires.
Grimpeur	/app/partnerships	Demandes émises et reçues.
Grimpeur	/app/places	Recherche et consultation du répertoire.
Grimpeur	/app/events	Consultation et inscription.
Grimpeur	/profile/me	Onglets et formulaires du profil.
Club	/app/events	Formulaire de publication et administration.
Club	/profile/me	Profil et compte club.

Rôle	Route ou état	Parcours
Grimpeur mobile	/app , largeur 360 px	Reflow du tableau de bord.
Grimpeur mobile	/profile/me , largeur 360 px	Reflow des onglets et formulaires.

L'échantillon couvre le processus complet de création de compte, de profil, de matching et d'événement retenu pour le BC02. Les états sans contenu et avec données sont exercés grâce à des comptes éphémères créés par le script.

3. Méthode combinée

L'audit ne confond pas score automatisé et conformité RGAA. Il combine :

- inspection sémantique des composants et pages ;
- tests axe dans Jest sur les contrôles partagés, les deux formulaires d'authentification et le formulaire événement ;
- test clavier du bouton d'affichage du mot de passe ;
- Lighthouse public sur trois pages de production ;
- Lighthouse authentifié sur dix états, avec seuil accessibilité bloquant de 100 % ;
- pilotage Chrome du lien d'évitement et de la préférence `prefers-reduced-motion` ;
- mesure du reflow à 360 px par comparaison de `scrollWidth` et `clientWidth` ;
- captures pleine page du tableau de bord et du profil mobile ;
- contrôle manuel des critères non automatisables sur le périmètre fonctionnel.

Commandes de reproduction :

```
npm test -- --runInBand
npm run accessibility:audit
npm run perf:audit
```

Le script `run-authenticated-accessibility.mjs` démarre Next.js avec MariaDB, crée un grimpeur et un club, contrôle les pages avec leurs cookies de session, produit les rapports et captures dans `.accessibility`, supprime les comptes, puis arrête le serveur. Le job GitHub Actions conserve ces preuves pendant 30 jours.

4. Écarts trouvés et corrections

Le premier audit n'a pas été présenté comme réussi. Il a détecté les écarts suivants :

ID	Écart initial	Mesure avant	Correction	Résultat après
A11 Y-01	Absence de lien d'évitement global.	Non mesuré	Premier élément focusable « Aller au contenu principal », cible focusable <code>#contenu-principal</code> .	Navigation clavier automatisée réussie.
A11 Y-02	Erreurs d'authentification et certains retours de profil non annoncés.	Retours seulement visuels	<code>role="alert"</code> , <code>role="status"</code> , <code>aria-live</code> , <code>aria-invalid</code> et <code>aria-describedby</code> .	Tests axe sans violation détectée.
A11 Y-03	Pas de prise en compte globale de la réduction des animations.	Défilement fluide et transitions actifs	Media query <code>prefers-reduced-motion: reduce</code> , animations et transitions ramenées à 0,01 ms, scroll automatique.	Style calculé <code>scroll-behavior: auto</code> .

ID	Écart initial	Mesure avant	Correction	Résultat après
A11 Y-04	Saut de titre de niveau 1 à niveau 3 sur le dashboard et les lieux.	Scores 98/100	<code>CardTitle</code> devient un titre de niveau 2 ; les titres internes restent de niveau 3.	100/100 sur les deux pages.
A11 Y-05	Badge gris : contraste 4,15:1 au lieu de 4,5:1.	Matching 96/100	Texte neutre assombri de <code>#65736b</code> à <code>#55645b</code> .	Contraste calculé 5,21:1 ; matching 100/100.
A11 Y-06	Texte de progression : contraste 3,12:1.	Profils 96/100	Vert foncé assombri de <code>#5f8f50</code> à <code>#376b31</code> .	Contraste calculé 5,22:1 ; profils 100/100.
A11 Y-07	Le bouton de déconnexion créait un document de 397 px à 360 px.	Échec du test de reflow	Commande mobile compacte de 44 x 44 px avec nom accessible et tooltip.	Dashboard : <code>scrollWidth=360</code> , <code>clientWidth=360</code> .
A11 Y-08	Les onglets du profil créaient un document de 566 px à 360 px.	Échec du test de reflow	Barre limitée à 100 % de son conteneur avec défilement interne contrôlé.	Profil : <code>scrollWidth=360</code> , <code>clientWidth=360</code> .
A11 Y-09	Seuil CI accessibilité fixé à 95 %.	Une régression mineure pouvait passer.	Seuil Lighthouse porté à 100 % pour la catégorie accessibilité.	CI bloquante sous 1,00.

Les ratios sont calculés selon la luminance relative utilisée par WCAG/RGAA. Les corrections sont appliquées aux tokens ou composants partagés afin d'éviter une correction locale fragile.

5. Grille par thème RGAA

Le RGAA 4.1.2 contient 106 critères répartis en 13 thèmes. Le tableau suivant qualifie chaque thème sur l'échantillon audité. « Non applicable » signifie que le prototype ne contient pas le type de contenu concerné ; cela ne vaut pas pour une future fonctionnalité qui l'introduirait.

Thème RGAA	Situation dans Spity	Contrôles et preuves	Résultat échantillon
1. Images	Logos décoratifs, images d'ambiance, avatar éventuel.	<code>alt=""</code> pour le décoratif ; nom accessible pour l'avatar porteur d'information ; texte utile hors des images de fond.	Conforme sur l'échantillon
2. Cadres	Aucun <code>iframe</code> dans le prototype.	Recherche du code et inspection DOM.	Non applicable
3. Couleurs	Badges, états, textes sur fonds clairs et sombres.	Les états possèdent un libellé ; ratios corrigés à 5,21:1 et 5,22:1 ; Lighthouse 100 %.	Conforme sur l'échantillon
4. Multimédia	Aucun média temporel audio ou vidéo.	Inspection du périmètre.	Non applicable
5. Tableaux	Aucun tableau de données dans les parcours retenus.	Les listes sont structurées en articles/cartes.	Non applicable
6. Liens	Navigation, fiches de lieux, agenda et actions de retour.	Intitulés explicites, focus visible, destination compréhensible hors contexte immédiat.	Conforme sur l'échantillon
7. Scripts	Filtres, onglets, formulaires, retours dynamiques.	Contrôles natifs, noms accessibles, clavier, <code>aria-live / role</code> , axe et test Puppeteer.	Conforme sur l'échantillon
8. Éléments obligatoires	Pages françaises générées par Next.js.	<code>lang="fr"</code> , métadonnées de titre/description, balisage valide contrôlé par Lighthouse.	Conforme sur l'échantillon
9. Structuration	Header, navigation, contenu principal, sections et titres.	Landmark <code>main</code> , navigation nommée, un <code>h1</code> , ordre <code>h1</code> puis <code>h2 / h3</code> corrigé.	Conforme sur l'échantillon

Thème RGAA	Situation dans Spity	Contrôles et preuves	Résultat échantillon
10. Présentation	Responsive, focus, animations et contenus masqués.	CSS séparée, focus visible, réduction de mouvement, zoom/reflow à 360 px sans débordement du document.	Conforme sur l'échantillon
11. Formulaires	Authentification, profils, filtres et événements.	Labels associés, fieldset/legend pour le rôle, autocomplete auth, erreurs reliées, validation Zod, boutons occupés avec <code>aria-busy</code> .	Conforme sur l'échantillon
12. Navigation	Header, barre d'onglets et accès au contenu.	Lien d'évitement, <code>aria-current</code> , ordre DOM logique, zones de scroll internes, commandes au moins 44 px.	Conforme sur l'échantillon
13. Consultation	Aucun délai, clignotement, rafraîchissement automatique ou document téléchargé.	Responsive portrait, réduction des animations, absence de limite de temps dans le parcours.	Conforme pour les critères applicables

6. Résultats automatisés du 20 juillet 2026

Lighthouse public en production

Page	Performance	Accessibilité	Bonnes pratiques	SEO
Accueil	99	100	100	100
Connexion	97	100	96	100
Inscription	99	100	96	100

Lighthouse authentifié

État	Score accessibilité
Dashboard grimpeur	100
Matching grimpeur	100
Demandes grimpeur	100
Lieux grimpeur	100
Événements grimpeur	100
Profil grimpeur	100
Événements club	100
Profil club	100
Dashboard grimpeur, viewport 360 x 800	100
Profil grimpeur, viewport 360 x 800	100

Contrôles d'interaction et de reflow

```
Lien d'évitement premier au focus : réussi
Cible #contenu-principal après activation : réussie
Préférence de réduction du mouvement : réussie
Dashboard 360 px : scrollWidth 360, aucun débordement
Profil 360 px : scrollWidth 360, aucun débordement
```

Les captures `dashboard-mobile.png` et `profile-mobile.png` sont générées dans l'artefact `.accessibility`. Leur inspection confirme l'absence de chevauchement incohérent ; la navigation et les onglets utilisent un scroll interne lorsque tous les items ne tiennent pas sur une seule ligne.

Preuve GitHub Actions

L'[exécution no 29743712530](#) est réussie sur le SHA `3779eee23e86f013c76e54f8f96a44a87a80b31c` :

- audit authentifié exécuté avec succès en 2 min 24 s ;
- job `Lighthouse thresholds` réussi sur les pages publiques ;
- artefact `accessibility-3779eee23e86f013c76e54f8f96a44a87a80b31c` de 1 509 342 octets ;
- rapports JSON, synthèse et deux captures mobiles conservés jusqu'au 19 août 2026.

Tests de composants

Les tests `jest-axe` couvrent :

- Input et Textarea avec erreurs ;
- toolbar de recherche et filtres ;
- formulaires connexion et inscription ;
- formulaire de création d'événement ;
- activation clavier de l'affichage du mot de passe.

Ils s'intègrent au seuil de couverture et au job `Quality gates`.

7. Limites et plan de maintien

Un score axe ou Lighthouse de 100 % ne contrôle pas tous les critères RGAA, notamment la pertinence éditoriale, la compréhension par des utilisateurs réels et l'interopérabilité complète avec les lecteurs d'écran. En conséquence :

- le résultat est formulé comme conformité de l'échantillon aux critères applicables vérifiés, pas comme déclaration légale de conformité globale du futur service ;
- un passage manuel NVDA + Firefox et VoiceOver + Safari reste requis avant une publication publique ;
- chaque nouveau composant interactif doit rejoindre les tests axe ou le scénario navigateur ;
- tout ajout de vidéo, iframe, tableau complexe, carte ou upload déclenchera la réouverture des thèmes actuellement non applicables ;
- les rapports CI sont conservés 30 jours et les résultats synthétiques sont inscrits dans ce dossier.

8. Conclusion

Le prototype répond au RGAA 4.1.2 sur l'échantillon fonctionnel BC02 vérifié : structure, formulaires, navigation clavier, annonces dynamiques, contrastes, réduction des animations et reflow mobile sont traités et testés. Les écarts détectés lors du premier passage ont été corrigés puis retestés à 100 % sur treize pages ou états publics et authentifiés.

La limite restante concerne la validation avec plusieurs technologies d'assistance et des utilisateurs en situation de handicap ; elle est inscrite au plan de recette avant mise en production.

Gestion des versions et déploiements progressifs - C2.2.4

1. Objectif et critères couverts

Ce document décrit la réponse de Spity à la compétence C2.2.4 : déployer chaque modification validée de façon progressive, vérifier sa fiabilité fonctionnelle et technique, tracer les évolutions et conserver une dernière version manipulable.

La grille d'évaluation attend :

- l'historique des différentes versions ;
- la dernière version fonctionnelle, fiable et viable ;
- un système de gestion de versions effectivement utilisé ;
- la traçabilité des évolutions du prototype ;
- un logiciel qu'un utilisateur peut manipuler en autonomie.

2. Convention de version

Spity utilise Semantic Versioning sous la forme `MAJEURE.MINEURE.CORRECTIF` :

Élément	Incrément	Exemple
MAJEURE	Rupture de compatibilité ou migration non rétrocompatible	1.0.0 vers 2.0.0
MINEURE	Nouvelle fonctionnalité compatible	0.1.0 vers 0.2.0
CORRECTIF	Correction compatible sans nouvelle capacité métier	0.1.0 vers 0.1.1

Tant que le périmètre MVP n'est pas stabilisé en production, la version majeure reste `0`. La source de vérité est `spity/package.json`. Une release stable exige simultanément :

1. une version stable dans `package.json` ;
2. une entrée datée correspondante dans `CHANGELOG.md` ;
3. un tag Git annoté `v<version>` sur le commit validé ;
4. une GitHub Release portant le même tag ;
5. deux images OCI applicative et migration portant la version, le SHA et un digest immuable.

Le script `npm run release:verify -- vX.Y.Z` bloque la publication si le tag, `package.json` et le changelog divergent.

3. Identifiants et traçabilité

Identifiant	Rôle	Immutabilité
Commit Git	Modification élémentaire et auteur	Immuable par SHA
Tag <code>vX.Y.Z</code>	Point de version stable	Immuable après publication
Entrée <code>CHANGELOG.md</code>	Évolutions utiles au lecteur	Versionnée dans Git

Identifiant	Rôle	Immutabilité
Image <code>sha-<SHA></code>	Résultat déployé automatiquement sur staging	Immuable
Image <code><X.Y.Z></code>	Version promue en production	Immuable par convention
Digest <code>sha256:...</code>	Preuve binaire de l'image exacte	Immuable par définition
Tag <code>staging</code>	Dernier commit <code>develop</code> validé	Mobile
Tag <code>latest</code>	Dernière release stable	Mobile, jamais utilisé seul pour un rollback

La route `GET /api/health` retourne `status`, `version` et `revision`. Elle permet donc de vérifier qu'une instance exécute bien le binaire attendu, en plus de tester sa connexion MariaDB.

4. Déploiement continu vers staging

Le job `Deploy verified staging images` de [.github/workflows/ci.yml](#) s'exécute après le succès des trois jobs CI sur chaque push `develop` :

1. construction des images `runner` et `migration` à partir du SHA contrôlé ;
2. injection de la version et du SHA dans l'image applicative ;
3. démarrage d'une MariaDB isolée ;
4. application des migrations avec l'image dédiée ;
5. démarrage de l'image applicative sous l'utilisateur non privilégié `nextjs` ;
6. smoke test de `/api/health` avec égalité stricte de la version et du SHA ;
7. publication des tags immuables `sha-<SHA>` et mobiles `staging` sur GHCR ;
8. conservation pendant 30 jours du JSON de santé et des inspections OCI ;
9. enregistrement du job dans l'environnement GitHub `staging`.

Un échec dans les tests, la migration, le démarrage, le contrat de santé ou la publication empêche le déploiement staging.

5. Promotion d'une release stable

Le workflow [.github/workflows/release.yml](#) est déclenché par un tag `vX.Y.Z` et comporte trois niveaux :

Niveau	Contrôles	Résultat
Validation	Cohérence de version, qualité, couverture, audit, build, MariaDB, accessibilité et Lighthouse	Candidat accepté ou publication bloquée
Staging de release	Récupération obligatoire des images <code>sha-<SHA></code> déjà déployées, nouvelle migration et smoke test	Tags <code>candidate-X.Y.Z</code> sur les mêmes images
Production	Promotion des mêmes digests, bundle, somme SHA-256 et GitHub Release	Tags <code>X.Y.Z</code> et <code>latest</code> , release téléchargeable

Aucune image n'est reconstruite entre le déploiement continu staging et la promotion stable. La seule opération autorisée est l'ajout de tags au digest déjà vérifié.

Le bundle de release contient :

- `docker-compose.production.yml` ;

- `.env.production.example` sans secret ;
- `DEPLOYMENT.md` ;
- `CHANGELOG.md` ;
- `release-manifest.json` avec version, tag, SHA, images et digests ;
- une somme SHA-256 du fichier compressé.

6. Contrôles de fiabilité

Axe	Contrôle bloquant	Preuve
Structure	ESLint et TypeScript sans erreur	Job de validation
Régression	100 % des tests et seuils de couverture atteints	Rapport de couverture
Données	Migrations sur MariaDB vide et parcours HTTP réels	Résultats TAP
Sécurité	Aucune vulnérabilité de production haute ou critique	Journal npm audit
Accessibilité	Audit public et authentifié sans violation détectée	Rapports Lighthouse et axe
Performance	Seuils Lighthouse du projet respectés	Rapports JSON
Exploitation	Conteneurs réels, migration, healthcheck et métadonnées exactes	JSON de santé et inspections OCI
Intégrité	Même digest entre staging, candidat et production	<code>release-manifest.json</code>

7. Historique et dernière version

L'historique fonctionnel lisible est tenu dans `CHANGELOG.md`. L'historique exhaustif reste disponible dans Git avec l'auteur, la date, le message conventionnel et le SHA de chaque modification.

Version	Date	SHA	État	Preuves
0.1.0	20 juillet 2026	<code>0bdd4e7a350094657b8037ee0714ca0ee0617310</code>	Dernière version stable du MVP BC02	CI, staging, release, bundle et manifeste validés

La version `0.1.0` couvre les parcours inscription, connexion, profil, lieux, matching, partenariats et événements. Le prototype peut être démarré localement avec Docker ou depuis ses images de release en suivant [spity/DEPLOYMENT.md](#).

Preuve du déploiement continu

L'[exécution CI no 29745444314](#) a validé et déployé le SHA de release sur staging le 20 juillet 2026.

Job	Résultat
<code>Quality gates</code>	Succès en 1 min 08 s : 85 tests, couverture, audit, lint, typage et build
<code>MariaDB integration tests</code>	Succès en 3 min 50 s : migrations, 11 résultats TAP et accessibilité authentifiée
<code>Lighthouse thresholds</code>	Succès en 1 min 28 s
<code>Deploy verified staging images</code>	Succès en 3 min 12 s : construction, migration, smoke test et publication GHCR

Job	Résultat
Artefact staging	staging-0bdd4e7a350094657b8037ee0714ca0ee0617310 , digest sha256:afcd504b63272840cd5aab52e3759dba39b2943480397f3c2a816c6d9302a1a2

Preuve de la promotion stable

L'exécution [Release no 29745993383](#) a publié [Spity v0.1.0](#) en 6 min 40 s.

Niveau	Résultat
Validation du candidat	Succès en 4 min 56 s
Promotion du candidat immuable	Succès en 1 min 01 s
Publication stable	Succès en 34 s
Image applicative	ghcr.io/dorianyloj/spity:0.1.0
Digest applicatif	sha256:bd110f5ade8f6014da186e38cda99e662bbf59575cca514322aa4bede860c105
Image de migration	ghcr.io/dorianyloj/spity-migrations:0.1.0
Digest de migration	sha256:641e3c74e82e9e9138034154b26a73f59830fc224e7708f213bc0c6b985c1ca7
Bundle	spity-0.1.0.tar.gz , 3 212 octets
Somme du bundle	sha256:31f2b4bb419e9767bc2d8eb258bed08b04a135516be3d3c5fd13bb848ba6a1e4 vérifiée après téléchargement

Les deux digests du manifeste correspondent aux images d'abord publiées sous le tag immuable `sha-0bdd4e7...`, puis promues sans reconstruction. Les trois artefacts de release sont conservés 90 jours ; le bundle et sa somme restent également attachés à la GitHub Release.

8. Retour arrière

Le retour arrière utilise le tag de version précédent ou, de préférence, son digest enregistré dans le manifeste :

1. suspendre la promotion et conserver les journaux ;
2. rétablir `IMAGE_TAG`, `APP_VERSION` et `APP_REVISION` de la dernière version fiable ;
3. redémarrer l'image sans reconstruction ;
4. vérifier la route de santé et les parcours critiques ;
5. restaurer la sauvegarde seulement si la migration n'est pas rétrocompatible ;
6. ouvrir une anomalie et rejouer tout le pipeline après correction.

Les suppressions de colonnes ou de données sont différées afin de conserver une fenêtre de compatibilité avec la version précédente.

9. Retours utilisateurs

Les contrôles automatisés prouvent la stabilité technique mais ne remplacent pas l'observation d'utilisateurs. Une session pilote doit réunir au moins trois profils : un grimpeur cherchant un partenaire, un grimpeur organisant une sortie et un représentant de club.

Chaque participant exécute sans assistance les tâches suivantes :

1. créer un compte et compléter son profil ;
2. trouver un partenaire compatible et envoyer une invitation ;
3. accepter une relation avec un second compte ;
4. créer un événement puis s'y inscrire ;
5. retrouver la version utilisée dans la réponse de santé fournie par l'opérateur.

Champ collecté	Contenu attendu
Session	Date, version, SHA et environnement
Profil	Rôle utile au test, sans donnée personnelle inutile
Tâche	Réussie sans aide, réussie avec aide ou échouée
Mesure	Durée, erreur rencontrée et étape de blocage
Retour	Commentaire reformulé et niveau de sévérité
Décision	Accepté, correction planifiée ou hors périmètre justifié

Aucun retour réel n'est inventé dans ce dossier. Cette preuve restera indiquée comme manquante jusqu'à l'exécution documentée d'une session avec des participants distincts du développeur.

10. État de couverture C2.2.4

Critère	Réalisation	État
Système de gestion de versions	Git, SemVer, tags, changelog et validateur	Réalisé
Évolutions tracées	Commits conventionnels et changelog 0.1.0	Réalisé
Déploiement progressif	CI vers staging puis promotion des mêmes digests	Réalisé et validé à distance
Dernière version fiable	Release v0.1.0 avec bundle et manifeste	Publiée et vérifiée
Manipulation autonome	Compose et guide de déploiement versionné	Réalisé techniquement
Performance auprès des utilisateurs	Protocole défini	Session pilote réelle à organiser

Cahier de recettes - C2.3.1

1. Objet et périmètre

Ce cahier vérifie la conformité du prototype Spity aux fonctions F01 à F10 définies dans le [périmètre fonctionnel](#). Il couvre les parcours fonctionnels des rôles grimpeur et club ainsi que les contrôles structurels, de sécurité et d'accessibilité demandés par la grille C2.3.1.

Version locale exécutée le 20 juillet 2026 :

Élément	Valeur
Branche	deveLop
SHA du lot de recette	2941a441657f85b3198b5e73f9e314a01221c977
Application	Spity 0.1.0
Runtime	Node.js 22, Next.js 16.2.10
Navigateur	Playwright 1.61.1, Chromium 149
Données	MariaDB 11.4 migrée avec Drizzle
Exécution	6 scénarios réussis sur 6, aucun ignoré, aucun instable, 18,4 s

2. Stratégie de recette

Critères d'entrée

- les dépendances sont installées depuis `package-lock.json` ;
- `DATABASE_URL` et `JWT_SECRET` sont définis ;
- MariaDB est saine et toutes les migrations Drizzle sont appliquées ;
- Chromium est installé par Playwright ;
- les fonctions F01 à F10 sont disponibles sur la branche à vérifier.

Règles d'exécution

1. La recette crée trois grimpeurs, un club et un compte d'inscription avec des adresses uniques.
2. Les scénarios s'exécutent séquentiellement avec un seul worker pour conserver le cycle métier.
3. Les actions sensibles sont réalisées par l'interface ; les comptes de préparation sont créés par les API publiques avec session réelle.
4. Chaque réponse inattendue, élément absent ou autorisation incorrecte fait échouer la commande.
5. Les comptes et leurs données liées sont supprimés avant et après l'exécution.
6. Un échec conserve capture, vidéo et trace ; chaque exécution produit aussi des rapports HTML, JSON et JUnit.

Commande locale :

```
npm run db:migrate
npm run test:acceptance
```

La CI exécute la même commande sur une MariaDB vide. Le job de staging dépend du succès de la recette : aucun déploiement `develop` ne peut donc contourner un échec fonctionnel.

3. Scénarios fonctionnels exécutés

ID	Fonctions	Préconditions et actions principales	Résultat attendu	Résultat obtenu
REC-F01-001	F01	Ouvrir l'inscription, soumettre un mot de passe faible, créer un grimpeur, terminer l'onboarding, se déconnecter puis se reconnecter.	Erreur accessible sur le mot de passe faible ; session créée ; route d'onboarding imposée ; déconnexion et reconnexion effectives.	Conforme
REC-F02-001	F02	Ouvrir la fiche d'un grimpeur, modifier sa bio, puis créer par l'interface le profil complet d'un club.	Valeurs existantes restituées ; modification persistée ; profil club et accès à l'application créés.	Conforme
REC-F03-F04-001	F03, F04	Combiner localité, discipline, niveau, disponibilité et environnement ; vérifier l'état vide ; envoyer deux demandes ; accepter la première et refuser la seconde.	Seul le profil compatible apparaît ; les deux décisions sont possibles et restent visibles dans l'historique émetteur.	Conforme
REC-F05-F08-001	F05, F06, F07, F08	Le club crée une initiation d'une place ; un grimpeur s'inscrit ; le second est bloqué ; le club voit le participant, augmente la capacité, puis suit inscription, désinscription et annulation.	Création et modification persistées ; capacité jamais dépassée ; liste privée exacte ; place libérée ; événement annulé et traçable.	Conforme
REC-F09-001	F09	Accéder anonymement au matching, appeler le matching avec un club, muter depuis une origine étrangère et envoyer un UUID invalide.	Redirection vers la connexion avec CSP ; réponses respectives <code>403</code> , <code>403</code> et <code>422</code> , sans donnée sensible.	Conforme
REC-F10-001	F10	Charger le matching à 360 px avec réduction des animations, atteindre le lien par Tab, l'activer par Entrée et mesurer le document.	Un seul <code>h1</code> , navigation nommée, contrôle focusable, aucune largeur parasite et champs nommés.	Conforme

4. Couverture des fonctions attendues

Fonction	Preuve navigateur	Preuves complémentaires	État
F01 Authentification	REC-F01-001	Tests de validation, session, quota et routes HTTP	Couverte
F02 Profils	REC-F02-001	Schémas Zod et intégration MariaDB des deux rôles	Couverte
F03 Matching	REC-F03-F04-001	Tests unitaires des combinaisons de filtres	Couverte
F04 Partenariats	REC-F03-F04-001	Intégration des conflits, droits et historique	Couverte
F05 Administration d'événements	REC-F05-F08-001	Validation des dates, statuts et propriété	Couverte
F06 Consultation et capacité	REC-F05-F08-001	Intégration de deux inscriptions concurrentes	Couverte
F07 Inscription et désinscription	REC-F05-F08-001	Contraintes uniques MariaDB et réactivation	Couverte
F08 Suivi club	REC-F05-F08-001	Contrôle API de confidentialité des participants	Couverte
F09 Sécurité	REC-F09-001	Audit OWASP, audit npm, headers et tests CSRF/quota	Couverte
F10 Accessibilité	REC-F10-001	Axe authentifié et Lighthouse à 100 %	Couverte sur le prototype

5. Contrôles fonctionnels, structurels et de sécurité

Type	Commande ou contrôle	Critère de succès	Résultat du 20 juillet 2026
Fonctionnel navigateur	<code>npm run test:acceptance</code>	6/6, aucun skip ou flaky	6/6 en 18,4 s
Fonctionnel HTTP/DB	<code>npm run test:integration</code>	Toutes les étapes TAP réussies	11 résultats réussis sur la release de référence
Structure	<code>npm run lint</code>	Aucune erreur ESLint	Réussi
Structure	<code>npm run typecheck</code>	Aucune erreur TypeScript	Réussi
Régression	<code>npm run test:coverage</code>	100 % tests ; seuils dépassés	86/86 ; lignes 96 %, branches 89,69 %, fonctions 100 %
Sécurité dépendances	<code>npm run security:audit</code>	Aucune alerte haute ou critique	Réussi ; 2 alertes modérées suivies
Construction	<code>npm run build</code>	Build production complet	28 routes générées
Accessibilité	<code>npm run accessibility:audit</code>	Aucune violation axe sur l'échantillon	Réussi sur la release de référence
Performance	<code>npm run perf:audit</code>	Performance >= 85 ; accessibilité = 100	Seuils réussis sur la release de référence

Les résultats « release de référence » correspondent à la CI et à la release `v0.1.0` documentées dans [C2.2.4](#). Le nouveau job de recette rejoue automatiquement son contrôle navigateur à chaque push et chaque pull request.

6. Résultat et décision

La recette locale du SHA `2941a44` est acceptée : toutes les fonctions F01 à F10 ont au moins un scénario nominal ou alternatif exécuté, les contrôles structurels sont réussis et les protections critiques refusent les accès invalides.

L'anomalie d'accessibilité détectée pendant la première exécution a été corrigée et retestée. Les échecs liés aux sélecteurs Playwright ont été qualifiés comme défauts du harnais, puis rendus déterministes. Le détail avant/après se trouve dans le [plan de correction C2.3.2](#).

Preuve distante

L'exécution [GitHub Actions no 29747713909](#) a validé le SHA `f384a21136fdaff73810cecefafb24951b82d42e` le 20 juillet 2026 :

Job	Résultat
<code>Quality gates</code>	Succès en 1 min 15 s
<code>MariaDB integration tests</code>	Succès en 4 min 01 s
<code>Lighthouse thresholds</code>	Succès en 1 min 46 s
<code>BC02 acceptance recipe</code>	Succès en 2 min 07 s
<code>Deploy verified staging images</code>	Succès en 3 min 19 s, déclenché seulement après les quatre portes précédentes
Artefact de recette	<code>acceptance-f384a21136fdaff73810cecefafb24951b82d42e</code> , 206 Ko, conservé 30 jours

Job	Résultat
Artefact staging	staging-f384a21136fdaff73810cecefab24951b82d42e , 3,58 Ko, conservé 30 jours

Cette exécution sur un runner et une base neuvs confirme les résultats locaux, la génération des preuves et le caractère bloquant de la recette avant staging.

Consolidation sur l'artefact standalone

L'exécution [GitHub Actions no 29750556481](#) a ensuite validé le SHA `689e59dccf15dc611dbddccea9cf81ccc187c40c` contre le build standalone réellement utilisé dans l'image Docker :

Job	Résultat distant
Quality gates	Succès en 1 min 10 s
MariaDB integration tests	Succès en 3 min 33 s
Lighthouse thresholds	Succès en 1 min 18 s
BC02 acceptance recipe	Succès en 1 min 48 s, build inclus, sans retry
Deploy verified staging images	Succès en 2 min 38 s
Artefact de recette	acceptance-689e59d... , 210 744 octets
Artefact staging	staging-689e59d... , 3 664 octets

Le run précédent `29749715001` avait échoué uniquement sur la recette et le staging avait été ignoré. Le passage au serveur standalone a donc été retesté sur un runner neuf tout en confirmant que la dépendance bloquante fonctionne dans les deux sens.

7. Preuves conservées

Preuve	Emplacement ou nom
Scénarios exécutables	spity/tests/acceptance/bc02-recipe.spec.ts
Configuration	spity/playwright.config.ts
Rapport local lisible	spity/playwright-report/index.html
Résultats structurés	spity/.acceptance-results/results.json et results.xml
Diagnostics d'échec	spity/test-results/ : capture, vidéo et trace
Artefact CI	acceptance-<SHA> , conservé 30 jours
Pipeline bloquant	Job BC02 acceptance recipe dans .github/workflows/ci.yml

Les rapports locaux sont régénérables et ignorés par Git pour éviter de versionner des fichiers volumineux ou dépendants de la machine. Les rapports de référence sont conservés par GitHub Actions.

Plan de correction des bogues - C2.3.2

1. Objectif

Ce registre qualifie les anomalies réellement détectées pendant le développement et la recette de Spity. Il relie chaque échec à son impact, sa cause, sa correction et son retest. Aucun bogue ni résultat utilisateur n'est inventé.

2. Méthode de traitement

Qualification

Sévérité	Définition	Délai cible
Bloquante	Empêche la CI, le déploiement ou un parcours critique sans contournement.	Correction avant toute livraison
Majeure	Altère une fonction métier importante ou son intégrité, avec contournement limité.	Lot courant
Mineure	Dégrade l'accessibilité, la preuve ou l'exploitation sans perte métier immédiate.	Lot courant ou suivant

La priorité combine la sévérité, la fréquence, le risque de régression et l'exposition utilisateur : **P0** immédiat, **P1** avant livraison, **P2** planifié.

Cycle

1. Conserver la commande, l'entrée et le résultat qui échoue.
2. Reproduire sur un environnement contrôlé.
3. Qualifier l'impact, la sévérité et la priorité.
4. Identifier la cause racine et le point de contrôle manquant.
5. Corriger sans réduire les seuils de qualité.
6. Ajouter ou renforcer un test automatisé.
7. Rejouer le test ciblé puis le pipeline complet.
8. Fermer uniquement lorsque le résultat attendu est obtenu et traçable par SHA.

3. Registre des anomalies traitées

ID	Anomalie et impact	Sévérité / priorité	Cause racine	Correction et SHA	Retest	État
BU G-TE ST-001	9,1 mm était découpé en deux articles ; diamètre et couleur du matériel étaient perdus.	Majeure / P1	Toutes les virgules étaient considérées comme séparateurs de liste.	Ne plus découper une virgule introduisant une décimale et préserver le texte des catégories inconnues. eb837f4	Cas Beal Joker corde 60 m 9,1 mm turquoise ; parseur à 100 % de couverture.	Fermée
BU G-CI-002	Les rapports Lighthouse existaient mais l'artefact CI ne les embarquait pas, supprimant une preuve du contrôle.	Majeure / P1	.lighthouseci est caché et l'upload excluait les fichiers cachés.	Ajouter include-hidden-files: true . 25d0de5	Artefact Lighthouse-25d0de5... produit et téléchargeable.	Fermée

ID	Anomalie et impact	Sévérité / priorité	Cause racine	Correction et SHA	Retest	État
BUG-MA TC H-003	Un profil ayant désactivé sa recherche restait contactable lors d'une nouvelle demande.	Majeure / P1	Le même accès public servait à l'éligibilité au matching et à la restitution de l'historique.	Utiliser un accès dédié qui exige <code>partnerSearch.enabled</code> tout en gardant l'accès public pour l'historique. <code>1db84b8</code>	CI <code>29739778393</code> , tests de matching et historique acceptée/refusée de REC-F03-F04-001.	Fermée
BUG-CI-004	GitHub annotait les jobs avec la dépréciation du runtime Node.js 20 des actions.	Mineure / P1	<code>setup-node@v4</code> et <code>upload-artifact@v4</code> ciblaient l'ancien runtime des actions.	Migrer les actions officielles vers <code>v6</code> . <code>8be8a29</code>	CI <code>29744313577</code> réussie sans annotation.	Fermée
BUG-A11 Y-005	Le titre visuel « Aucun profil ne correspond » était annoncé comme paragraphe, ce qui dégradait la navigation par titres.	Mineure / P1	Le composant partagé <code>EmptyState</code> utilisait un élément <code>p</code> pour son titre.	Remplacer le paragraphe par un <code>h2</code> et ajouter un test de rôle/niveau. <code>2941a44</code>	Test Jest dédié et REC-F03-F04-001 réussis ; 86/86 tests.	Fermée
BUG-TEST-006	L'ajout de Playwright faisait collecter le fichier de recette par Jest et bloquait <code>npm run quality</code> .	Bloquant / P0	Le motif Jest excluait l'intégration mais pas <code>tests/acceptance</code> .	Exclure explicitement le dossier Playwright de Jest. <code>2941a44</code>	Avant : 13 suites réussies et 1 suite en erreur ; après : 13/13 suites et 86/86 tests.	Fermée
BUG-TEST-007	Une exécution de recette sur un commit documentaire a échoué et a bloqué le staging, alors que le même code produit passait localement et dans la CI précédente.	Bloquant / P0	Le job ciblait <code>next dev</code> sur un cache froid et compilait les routes pendant la recette au lieu de tester l'artefact livré.	Construire le mode standalone, copier ses assets puis exécuter Playwright contre <code>node server.js</code> . Utiliser <code>localhost</code> en CI afin que le cookie <code>Secure</code> reste valide sur la boucle locale. <code>689e59d</code>	6/6 localement en 14,5 s sur le standalone ; CI <code>29750556481</code> et staging réussis sans retry.	Fermée

4. Analyse des échecs de la recette Playwright

Échec observé	Qualification	Amélioration appliquée	Prévention
« Mot de passe » ciblait le champ et le bouton d'affichage.	Harnais, sans défaut produit.	Ciblage explicite de <code>#register-password</code> et <code>#login-password</code> .	Utiliser rôle + nom seulement lorsqu'ils identifient un élément unique.
<code>getByRole('alert')</code> trouvait aussi l'annonceur de route Next.js.	Harnais, sans défaut produit.	Assertion sur l'identifiant de l'erreur rattachée au champ.	Préférer la relation champ-erreur à un rôle global non unique.
Deux liens « Entrer dans l'app » étaient présents après onboarding.	Harnais ; les deux commandes ont la même destination valide.	Sélection déterministe du premier raccourci visible.	Limiter les assertions strictes aux commandes uniques ou les cadrer par région.
L'état vide n'était pas trouvé comme titre.	Défaut produit BUG-A11Y-005.	Titre partagé rendu en <code>h2</code> .	Test unitaire sémantique et scénario navigateur maintenus dans la CI.
Jest chargeait la suite Playwright.	Défaut d'outillage BUG-TEST-006.	Séparation explicite des répertoires de tests.	<code>npm run quality</code> et <code>npm run test:acceptance</code> sont deux portes CI indépendantes.

Échec observé	Qualification	Amélioration appliquée	Prévention
La recette distante 29749715001 a échoué sur un SHA ne modifiant pas l'application.	Harnais BUG-TEST-007 ; le staging a été correctement ignoré.	Remplacement du serveur de développement par le build standalone utilisé dans Docker.	Aucun retry ajouté ; la CI suivante doit réussir les quatre portes avant de reconstruire le staging.
Le standalone renvoyait 401 après inscription sur 127.0.0.1.	Harnais ; le cookie de production est volontairement Secure.	URL de boucle locale CI alignée sur localhost, sans affaiblir le cookie ni modifier la production.	Test local avec NODE_ENV=production, puis exécution distante sur un runner neuf.

Ces échecs n'ont pas été masqués par des délais plus longs, des retries ou une baisse de seuil. Chaque cause a été corrigée au niveau du produit ou du harnais, puis la recette complète a été rejouée sans test ignoré ni instable.

5. Preuves avant et après

Contrôle	Avant	Après
Recette navigateur	Échecs successifs conservant captures, vidéos et traces.	6/6 en 14,5 s sur le build standalone ; aucun retry ni skip.
Structure de l'état vide	Titre exposé comme paragraphe.	h2 vérifié par Jest et Playwright.
Coexistence Jest/Playwright	1 suite en erreur lors de npm run quality.	13/13 suites, 86/86 tests.
Couverture	Seuils maintenus.	Lignes 96 %, branches 89,69 %, fonctions 100 %.
Construction	Non exécutée après le premier échec Jest.	Build Next.js réussi, 28 routes générées.
Sécurité des dépendances	Seuil inchangé.	0 haute, 0 critique ; 2 modérées documentées.

6. Risques suivis et décision

Deux avis PostCSS modérés restent ouverts dans une dépendance embarquée par Next.js. La correction automatique proposée impose un downgrade majeur incohérent ; le risque est accepté temporairement selon l'analyse [OWASP C2.2.3](#), avec une CI bloquante dès le niveau haut.

La session pilote avec des utilisateurs distincts du développeur reste à organiser. Elle pourra ouvrir de nouvelles anomalies d'usage ; celles-ci devront reprendre le même cycle de qualification, correction et retest.

À la date du 20 juillet 2026, aucune anomalie bloquante ou majeure connue de la recette F01 à F10 ne reste ouverte. La correction est considérée conforme lorsque le job distant de recette et les trois portes existantes sont réussis avant staging.

L'exécution [GitHub Actions no 29750556481](#) constitue le retest de BUG-TEST-007 : qualité, intégration MariaDB, Lighthouse, recette standalone et staging sont tous réussis sur le SHA 689e59dccf15dc611dbddccea9cf81ccc187c40c.

Manuel de déploiement de Spity - C2.4.1

Identification du document

Champ	Valeur
Produit	Spity
Version documentée	0.1.0 et versions ultérieures compatibles
Public	Développeur, exploitant, évaluateur RNCP
Environnements	Développement local, recette CI, staging et production Docker
Sources exécutables	spity/docker-compose.yml , spity/docker-compose.production.yml , spity/DEPLOYMENT.md
Compétence	C2.4.1 - documentation technique d'exploitation
Dernière vérification	20 juillet 2026

1. Objet et périmètre

Ce manuel permet d'installer Spity depuis les sources, de promouvoir une release immuable en production, de contrôler son bon fonctionnement et de revenir à une version précédente. Il couvre l'application Next.js et sa base MariaDB.

Les données de démonstration sont strictement réservées aux environnements locaux ou éphémères. Elles ne doivent jamais être chargées sur une base de production.

2. Architecture déployée

```
Navigateur
|
| HTTPS
v
Reverse proxy / terminaison TLS
|
| HTTP sur 127.0.0.1:3000
v
Application Next.js (conteneur non-root)
|
| DATABASE_URL sur le réseau Docker privé
v
MariaDB 11.4 + volume persistant

Image de migration Drizzle -- exécution ponctuelle --> MariaDB
```

En production, le port applicatif est lié à l'interface locale et MariaDB ne publie aucun port hôte. Le reverse proxy HTTPS est donc l'unique entrée publique. La migration est un service ponctuel distinct : son succès est obligatoire avant le démarrage de la nouvelle version applicative.

3. Choix technologiques

Couche	Technologie ou langage	Choix d'exploitation
Langage	TypeScript en mode strict	Un langage partagé côté interface et serveur limite les divergences de contrats et rend le typage vérifiable par CI.
Application	Next.js App Router et React	Les pages, composants serveur et Route Handlers sont construits et livrés dans un même artefact.
Exécution	Node.js 22	Version fixée par <code>.nvmrc</code> , <code>package.json</code> et l'image Docker pour rendre les builds reproductibles.
Validation	Zod	Les entrées et réponses métier sont contrôlées à l'exécution en complément du typage TypeScript.
Accès aux données	Drizzle ORM	Le schéma typé et les migrations versionnées évitent les modifications manuelles non traçables.
Base de données	MariaDB 11.4	Base relationnelle adaptée aux comptes, relations de partenariat, inscriptions et capacités d'événements.
Présentation	Tailwind CSS et composants internes	Le design system reste dans le dépôt et peut être audité pour le responsive et l'accessibilité.
Conteneurs	Docker multi-stage et Compose v2	Les images application/migration sont séparées ; le conteneur applicatif final fonctionne avec un utilisateur non-root.
Livraison	GitHub Actions et GHCR	Chaque image staging est identifiée par le SHA Git ; une release promeut le candidat déjà construit, puis génère un bundle et un manifeste.
Contrôle	Jest, Playwright, axe, Lighthouse	La publication est précédée de portes qualité unitaires, fonctionnelles, sécurité, accessibilité et performance.

Les décisions détaillées d'architecture se trouvent dans [l'architecture du prototype](#) et le protocole de livraison dans [la gestion des versions](#).

4. Prérequis

4.1 Depuis les sources

- Linux, macOS ou Windows avec un environnement de terminal compatible ;
- Node.js `22.x` et npm `10+` ;
- Docker Engine 24+ avec Docker Compose v2 ;
- ports locaux `3000` et `3306` disponibles ;
- Git pour récupérer et versionner les sources.

Contrôler les versions :

```
node --version
npm --version
docker --version
docker compose version
```

4.2 Depuis une release

- Docker Engine 24+ et Docker Compose v2 ;
- accès en lecture aux images GHCR privées ou publiques de Spity ;
- `curl`, `jq`, `sha256sum`, `tar` et `openssl` ;
- nom DNS, certificat TLS et reverse proxy pour l'exposition publique ;
- stratégie de sauvegarde chiffrée hors du serveur.

5. Installation de développement

Depuis la racine du dépôt :

```
cd spity
cp .env.example .env.local
```

Remplacer les secrets d'exemple dans `.env.local`. Un secret JWT peut être généré avec :

```
openssl rand -base64 64
```

Démarrer MariaDB, installer exactement les dépendances verrouillées, appliquer les migrations puis lancer l'application :

```
docker compose --env-file .env.local up -d mariadb
npm ci
npm run db:migrate
npm run dev
```

Contrôler :

```
docker compose --env-file .env.local ps
curl --fail http://localhost:3000/api/health
```

L'application est accessible sur <http://localhost:3000>. Pour ouvrir phpMyAdmin localement :

```
docker compose --env-file .env.local --profile tools up -d
```

L'outil est alors disponible sur <http://localhost:8083> par défaut. Il ne fait pas partie de la production.

5.1 Données de démonstration

Après les migrations, charger le jeu de démonstration uniquement en local :

```
npm run db:seed
```

Le script réinitialise les lignes associées à ses comptes de démonstration. Il est donc interdit sur une base contenant de vraies données.

5.2 Arrêt local

```
docker compose --env-file .env.local down
```

Cette commande conserve le volume. Ne pas ajouter `--volumes` si les données doivent être conservées.

6. Déploiement d'une release en production

La release GitHub fournit une archive `spity-VERSION.tar.gz`, sa somme SHA-256, un manifeste, le Compose de production, un modèle d'environnement et une procédure autonome.

6.1 Vérifier et extraire le bundle

```
sha256sum --check spity-0.1.0.tar.gz.sha256
tar --extract --gzip --file spity-0.1.0.tar.gz
cd spity-0.1.0
```

Adapter la version. Le `release-manifest.json` doit contenir le tag, le SHA Git, les images et leurs digests attendus. En cas d'écart, interrompre le déploiement.

6.2 Créer la configuration secrète

```
cp .env.production.example .env.production
chmod 600 .env.production
```

Variables obligatoires :

Variable	Usage	Règle
<code>MARIADB_ROOT_PASSWORD</code>	administration et sauvegarde	Secret dédié, jamais commité.
<code>MARIADB_PASSWORD</code>	compte SQL applicatif	Secret différent du compte root.
<code>DATABASE_URL</code>	connexion de l'application et des migrations	Mot de passe applicatif encodé comme composant d'URL.
<code>JWT_SECRET</code>	signature des sessions	Au moins 64 octets aléatoires.
<code>SPITY_IMAGE</code>	image applicative	<code>ghcr.io/dorianyloj/spity</code> .
<code>SPITY_MIGRATION_IMAGE</code>	image des migrations	<code>ghcr.io/dorianyloj/spity-migrations</code> .
<code>IMAGE_TAG</code>	tag Docker immuable	Version stable, jamais <code>latest</code> seul.
<code>APP_VERSION</code>	version annoncée par <code>/api/health</code>	Identique à <code>package.json</code> et au tag sans <code>v</code> .
<code>APP_REVISION</code>	révision annoncée par <code>/api/health</code>	SHA Git complet du tag.
<code>APP_PORT</code>	port local derrière le proxy	<code>3000</code> par défaut.

Valider la configuration sans afficher les valeurs dans un ticket ou un document partagé :

```
docker compose --env-file .env.production -f docker-compose.production.yml config --quiet
```

6.3 Tirer les images et démarrer MariaDB

```
docker login ghcr.io
docker compose --env-file .env.production -f docker-compose.production.yml pull app migrate
docker compose --env-file .env.production -f docker-compose.production.yml up -d mariadb
docker compose --env-file .env.production -f docker-compose.production.yml ps mariadb
```

Attendre l'état `healthy`. Si MariaDB reste indisponible, consulter ses journaux et ne pas lancer la migration.

6.4 Sauvegarder

```
docker compose --env-file .env.production -f docker-compose.production.yml exec -T mariadb \
  sh -c 'exec mariadb-dump -uroot -p"$MARIADB_ROOT_PASSWORD" "$MARIADB_DATABASE" \
  > "backup-before-$IMAGE_TAG.sql"
test -s "backup-before-$IMAGE_TAG.sql"
```

Le `sh -c` est volontaire : les variables sont évaluées à l'intérieur du conteneur, où Compose les a injectées. Chiffrer et transférer la sauvegarde sur un support distinct avant de poursuivre.

6.5 Migrer et démarrer

```
docker compose --env-file .env.production -f docker-compose.production.yml \
  --profile migration run --rm --no-build migrate
docker compose --env-file .env.production -f docker-compose.production.yml \
  up -d --no-build app
```

Un échec de migration bloque la promotion. Ne pas contourner cette étape et ne pas exécuter `db:push` en production.

6.6 Contrôler la version

```
health="$(curl --fail --silent http://127.0.0.1:3000/api/health)"
printf '%s\n' "$health" | jq .
printf '%s\n' "$health" | jq --exit-status \
  --arg version "$APP_VERSION" \
  --arg revision "$APP_REVISION" \
  '.status == "ok" and .version == $version and .revision == $revision'
docker compose --env-file .env.production -f docker-compose.production.yml ps
```

Le contrôle fonctionnel public porte ensuite sur l'accueil, la connexion, le profil, les lieux, le matching et les événements. La promotion ne peut être déclarée réussie qu'après validation de l'URL HTTPS et du certificat.

7. Reverse proxy et sécurité d'exposition

Le reverse proxy doit :

- terminer TLS avec un certificat valide ;
- rediriger HTTP vers HTTPS ;
- transmettre `Host`, `X-Forwarded-For` et `X-Forwarded-Proto` ;

- appliquer une limite de taille et des délais adaptés ;
- ne jamais exposer MariaDB ni le port Docker interne ;
- conserver les en-têtes de sécurité émis par Next.js.

Le conteneur applicatif est déjà lié à `127.0.0.1`. Une modification vers `0.0.0.0` doit faire l'objet d'une revue de sécurité.

8. Exploitation et supervision

```
# Santé et état
curl --fail --silent http://127.0.0.1:3000/api/health | jq .
docker compose --env-file .env.production -f docker-compose.production.yml ps

# Journaux
docker compose --env-file .env.production -f docker-compose.production.yml logs --tail=200 app
docker compose --env-file .env.production -f docker-compose.production.yml logs --tail=200 mariadb

# Redémarrage applicatif
docker compose --env-file .env.production -f docker-compose.production.yml restart app
```

Une supervision externe doit au minimum vérifier la disponibilité HTTPS et `/api/health`. Ne jamais consigner les cookies, le JWT, `DATABASE_URL` ou les mots de passe dans les preuves d'incident.

9. Retour arrière et restauration

Le premier choix est un retour arrière applicatif : remettre le tag, la version et le SHA précédents dans `.env.production`, puis tirer et relancer `app`. Cette opération évite de toucher aux données si les migrations restent rétrocompatibles.

```
docker compose --env-file .env.production -f docker-compose.production.yml pull app
docker compose --env-file .env.production -f docker-compose.production.yml up -d --no-build app
```

Restaurer la base uniquement si l'ancienne application ne peut pas exploiter le schéma migré, avec validation du responsable et acceptation de la perte des écritures postérieures à la sauvegarde :

```
docker compose --env-file .env.production -f docker-compose.production.yml stop app
cat "backup-before-VERSION.sql" | \
  docker compose --env-file .env.production -f docker-compose.production.yml exec -T mariadb \
  sh -c 'exec mariadb -uroot -p"$MARIADB_ROOT_PASSWORD" "$MARIADB_DATABASE"'
docker compose --env-file .env.production -f docker-compose.production.yml up -d --no-build app
```

Après retour arrière, contrôler la santé, rejouer les parcours critiques et inscrire l'incident dans le [plan de correction](#).

10. Diagnostic

Situation	Preuve à relever	Décision
MariaDB <code>unhealthy</code>	<code>ps mariadb</code> , <code>logs mariadb</code> , espace disque	Corriger l'infrastructure sans supprimer le volume.
Migration en erreur	sortie du service <code>migrate</code> , migration concernée	Bloquer la nouvelle version ; corriger par une nouvelle migration.
Application en redémarrage	<code>logs app</code> , <code>DATABASE_URL</code> , santé MariaDB	Corriger la configuration ou revenir à l'image précédente.

Situation	Preuve à relever	Décision
Santé OK mais version fausse	JSON de santé, manifeste et <code>.env.production</code>	Interrompre la promotion ; redéployer le tag attendu.
Échec uniquement via HTTPS	certificat et journaux du proxy	Maintenir l'application locale et corriger le proxy.

11. Checklist de déploiement

- ☐ Bundle et somme SHA-256 vérifiés.
- ☐ Tag, SHA et digests comparés au manifeste.
- ☐ Secrets propres à l'environnement et fichier protégé.
- ☐ Configuration Compose valide.
- ☐ MariaDB saine et sauvegarde non vide externalisée.
- ☐ Migration ponctuelle terminée avec succès.
- ☐ Application saine avec version et révision exactes.
- ☐ HTTPS, certificat et parcours critiques vérifiés.
- ☐ Résultat, opérateur, heure et éventuel incident consignés.

12. Preuves reproductibles

La configuration d'environnement et les contrôles qualité sont décrits dans [C2.1.1](#). La chaîne distante, les images immuables et la release `v0.1.0` sont tracées dans [C2.2.4](#). La procédure courte réellement embarquée avec la release est [spity/DEPLOYMENT.md](#).

Relevé de vérification du 20 juillet 2026

Contrôle exécuté	Résultat
Résolution des liens relatifs des six documents d'exploitation	Conforme, aucun lien local manquant.
<code>docker compose --env-file .env.example -f docker-compose.yml config --quiet</code>	Succès.
<code>docker compose --env-file .env.production.example -f docker-compose.production.yml config --quiet</code>	Succès.
Sauvegarde avec <code>mariadb-dump</code> et variables évaluées dans le conteneur	Succès, fichier SQL non vide de 33 005 octets sur la base locale de démonstration.
<code>npm run release:verify -- v0.1.0</code>	Succès, tag et version <code>0.1.0</code> concordants.
<code>GET /api/health</code> sur l'application locale	HTTP réussi, <code>status</code> égal à <code>ok</code> , version <code>0.1.0</code> .
CI et staging du SHA <code>689e59d</code>	Run <code>29750556481</code> réussi : cinq jobs verts, images standalone migrées et testées avant publication.

Les secrets et le contenu de la sauvegarde ne sont pas intégrés au dépôt. La restauration, destructive par nature, est documentée mais n'a pas été exécutée sur la base de travail.

Manuel d'utilisation de Spity - C2.4.1

Identification du document

Champ	Valeur
Produit	Spity
Version documentée	0.1.0
Public	Grimpeurs, responsables de club et évaluateurs
Périmètre	Prototype BC02, fonctions F01 à F10
Recette associée	Cahier de recettes
Dernière vérification	20 juillet 2026

1. Présentation

Spity rassemble dans une même application les profils d'escalade, la recherche de partenaires, les demandes de mise en relation, les événements communautaires et un répertoire de salles, falaises et clubs.

Deux rôles existent :

Rôle	Fonctions principales
Grimpeur	Gérer son profil et son matériel, filtrer les partenaires, envoyer ou traiter des demandes, consulter les lieux, s'inscrire aux événements.
Club	Gérer sa fiche, consulter les lieux, créer et administrer ses événements, suivre les participants.

Le rôle choisi à l'inscription détermine les fonctions accessibles. Il n'est pas modifiable depuis l'interface du prototype.

2. Accès et navigation

L'application nécessite un navigateur récent avec JavaScript et les cookies activés. Les parcours ont été vérifiés sur Chromium en affichage bureau et mobile.

Une fois connecté, la navigation principale contient :

- **Feed** : tableau de bord et synthèse du compte ;
- **Partenaires** : annuaire filtrable, réservé aux grimpeurs ;
- **Demandes** : invitations reçues et historique, réservé aux grimpeurs ;
- **Lieux** : salles, falaises, voies et clubs ;
- **Événements** : inscriptions grimpeur ou gestion club ;
- **Profil** : informations personnelles, pratique et matériel ;
- **Déconnexion** : fin de session.

Tous les liens, champs et boutons des parcours critiques sont utilisables au clavier avec **Tab**, **Maj+Tab**, **Entrée** et **Espace**. Le focus visible indique l'élément actif. Les messages de réussite ou d'erreur sont annoncés après les actions.

3. Comptes de démonstration locaux

Après exécution locale de `npm run db:seed`, les comptes suivants utilisent le mot de passe commun `SpityDemo2026!` :

Compte	Rôle	Usage de démonstration
<code>lina.demo@spity.local</code>	Grimpeur	Profil complet, matériel, événements et relation existante.
<code>nassim.demo@spity.local</code>	Grimpeur	Deuxième profil et relation avec Lina.
<code>camille.demo@spity.local</code>	Grimpeur	Profil supplémentaire pour les filtres de matching.
<code>club.demo@spity.local</code>	Club	Création et gestion d'événements.

Ces identifiants sont publics et destinés exclusivement à une base de démonstration locale ou éphémère. Ils ne doivent exister dans aucun environnement contenant de vraies données.

4. Créer un compte et se connecter

4.1 Inscription

- Ouvrir `/register` ou sélectionner **Créer un compte** depuis la connexion.
- Saisir une adresse e-mail valide.
- Créer un mot de passe d'au moins 8 caractères, avec une majuscule, une minuscule, un chiffre et un caractère spécial. La limite technique est de 72 octets.
- Choisir **Grimpeur** ou **Club**.
- Sélectionner **Créer mon compte**.

Une adresse déjà utilisée est refusée. Après création, Spity ouvre automatiquement l'étape de profil.

4.2 Première configuration grimpeur

- Choisir au moins une discipline : bloc, voie ou trad.
- Indiquer le niveau correspondant, de `4a` à `8c+` selon les options proposées.
- Sélectionner le matériel de base disponible.
- Valider avec **Créer le profil**.
- Utiliser **Entrer dans l'app**.

Les informations publiques et les préférences détaillées peuvent être complétées ensuite dans **Profil**.

4.3 Première configuration club

- Saisir le nom du club.
- Ajouter la localisation.
- Ajouter, si disponible, le numéro d'affiliation FFME.
- Rédiger une courte présentation.
- Valider avec **Créer le profil**, puis **Entrer dans l'app**.

4.4 Connexion et déconnexion

Sur `/login`, saisir l'e-mail et le mot de passe puis sélectionner **Se connecter**. Une session valide redirige vers le tableau de bord. Le bouton **Déconnexion** supprime la session et renvoie à la page de connexion.

Le prototype ne fournit pas encore de parcours autonome « mot de passe oublié ». Un compte de démonstration peut être recréé en rechargeant le jeu local ; un vrai compte nécessite actuellement une intervention de maintenance.

5. Gérer un profil grimpeur

Ouvrir **Profil**. Quatre onglets sont proposés.

5.1 Aperçu

L'aperçu affiche la fiche publique, la progression du profil, les disponibilités, les objectifs et les préférences de partenaire.

Pour modifier la fiche :

1. saisir un nom affiché, une localisation et éventuellement l'URL HTTPS d'une photo ;
2. choisir l'environnement principal et rédiger une bio ;
3. cocher les disponibilités et objectifs ;
4. sélectionner **Mettre à jour la fiche publique**.

Dans **Préférences de matching**, activer ou désactiver la recherche, choisir le niveau recherché et le style de session, ajouter une note, puis sélectionner **Enregistrer les préférences**. Un profil dont la recherche est désactivée n'est pas proposé aux autres grimpeurs.

5.2 Pratique

1. Cocher les disciplines pratiquées.
2. Choisir un niveau pour chaque discipline.
3. Enregistrer les modifications.

Ces données alimentent les filtres de matching. Une cotation incohérente ou un formulaire incomplet est refusé avant enregistrement.

5.3 Matériel

L'inventaire détaillé est réservé aux grimpeurs.

Saisie assistée :

1. saisir une liste dans **Liste libre**, par exemple 2 dégaines Petzl; casque Mammut; corde 70 m 9.5 mm ;
2. sélectionner **Analyser** ;
3. vérifier et corriger chaque proposition ;
4. sélectionner **Enregistrer** sur les éléments à conserver.

Saisie manuelle :

1. choisir la catégorie et la quantité ;
2. renseigner au minimum le modèle ou le nom ;
3. compléter marque, couleur, taille, longueur ou diamètre lorsque ces champs s'appliquent ;
4. choisir l'état et indiquer si l'objet est disponible pour un partenaire ;
5. enregistrer.

Les boutons **Modifier** et **Supprimer** de chaque ligne permettent ensuite d'entretenir l'inventaire. Vérifier physiquement le matériel avant une sortie : Spity consigne une déclaration, pas un contrôle de sécurité.

5.4 Compte

L'onglet récapitule l'adresse e-mail et le rôle. Le rôle ne peut pas être transformé depuis l'interface. La suppression et l'export autonome du compte ne sont pas encore disponibles dans ce MVP.

6. Trouver un partenaire

La rubrique **Partenaires** est réservée au rôle grimpeur.

1. Utiliser **Nom ou localisation** pour une recherche textuelle.
2. Combiner si nécessaire les filtres **Discipline**, **Niveau**, **Disponibilité** et **Environnement**.
3. Lire les informations du profil correspondant.
4. Sélectionner **Envoyer une demande** sur le profil souhaité.

Le bouton est remplacé par l'état de la relation lorsqu'une demande existe déjà. Si aucun résultat ne correspond, sélectionner **Réinitialiser les filtres** pour revenir à l'annuaire complet.

Spity exclut le compte connecté de ses propres résultats et n'affiche que les grimpeurs ayant activé leur recherche de partenaire.

7. Traiter les demandes

Ouvrir **Demandes**.

- Dans les demandes reçues, sélectionner **Accepter** pour créer la relation ou **Refuser** pour la clôturer.
- Les demandes envoyées restent visibles avec leur état : en attente, acceptée ou refusée.
- Une même paire de comptes ne peut pas créer plusieurs demandes actives concurrentes.

Le demandeur voit le nouvel état après actualisation de la page. Une demande refusée n'établira aucune relation.

8. Utiliser les événements

8.1 Parcours grimpeur

1. Ouvrir **Événements**.
2. Consulter le type, l'organisateur, la date, le lieu et le nombre de places.
3. Sélectionner **S'inscrire**.
4. Vérifier le message **Inscription confirmée** et le badge **Inscrit**.
5. Sélectionner **Annuler mon inscription** pour libérer la place.

Le bouton d'inscription est désactivé lorsque la capacité est atteinte. Un événement annulé reste identifiable mais n'accepte plus d'inscription.

8.2 Parcours club

1. Ouvrir **Événements**, puis **Nouvel événement**.
2. Renseigner titre, type, description, lieu, début, fin et capacité.
3. Sélectionner **Publier**.
4. Utiliser **Modifier** sur un événement appartenant au club pour ajuster ses informations.
5. Consulter la liste **Participants** sur la carte de l'événement.
6. Utiliser **Annuler l'événement** si nécessaire.

La date de fin doit être postérieure au début et la capacité doit être positive. Un club ne peut modifier ou annuler que ses propres événements.

9. Consulter les lieux

Ouvrir **Lieux**, puis :

1. filtrer par type de lieu, discipline et état ;
2. utiliser **Nom, ville, voie, service...** pour rechercher un contenu ;
3. ouvrir une fiche de salle ou de falaise pour consulter ses informations ;
4. consulter, pour une falaise, les secteurs, voies, cotations, hauteurs et états disponibles ;
5. consulter les fiches de clubs présentes dans le répertoire.

Le bloc **Carte à venir** est un emplacement visuel : aucune carte interactive ni géolocalisation n'est disponible dans la version **0.1.0**.

10. Tableau de bord et limites du prototype

Le **Feed** présente un tableau de bord avec des statistiques du compte, des aperçus de profil, de matériel et d'événements. Les publications de démonstration sont affichées pour illustrer le futur réseau social.

Dans la version **0.1.0** :

- la création de publication, les mentions « J'aime » et les commentaires du Feed ne sont pas persistants ;
- la carte des lieux n'est pas interactive ;
- l'export, la suppression autonome du compte et la réinitialisation du mot de passe ne sont pas exposés dans l'interface ;
- les topos collaboratifs, votes de cotation et signalements en temps réel restent hors du prototype démontrable ;
- les notifications externes par e-mail ou mobile ne sont pas envoyées.

Les parcours réellement couverts et automatisés sont listés dans le [cahier de recettes F01 à F10](#).

11. Messages et résolution des erreurs

Message ou situation	Cause probable	Action utilisateur
Identifiants invalides	E-mail ou mot de passe incorrect	Vérifier la saisie ; ne pas multiplier les essais rapides.
Formulaire refusé	Champ obligatoire ou format incorrect	Lire le message associé au champ et corriger sa valeur.
Accès renvoyé vers /login	Session absente ou expirée	Se reconnecter.
Accès renvoyé vers l'onboarding	Profil du rôle incomplet	Terminer la création du profil.
Fonction non autorisée	Rôle incompatible ou ressource appartenant à un autre compte	Utiliser le bon rôle ou revenir à son propre contenu.
Aucune place disponible	Capacité atteinte	Choisir un autre événement ou attendre une annulation.
Action sans confirmation	Réseau ou serveur indisponible	Ne pas répéter immédiatement ; actualiser et vérifier l'état avant de réessayer.

Ne jamais communiquer son mot de passe ou son cookie de session dans une capture ou un ticket. Pour signaler un défaut, joindre l'heure, la page, le rôle, les étapes, le résultat observé et, si possible, une capture sans donnée sensible.

12. Parcours de démonstration jury

Le jeu local permet un parcours court et reproductible :

1. se connecter avec Lina et présenter le profil, le matériel, les filtres de partenaires, les lieux et une inscription ;
2. se déconnecter puis ouvrir le compte club ;
3. créer un événement futur avec une capacité de **1** ;
4. revenir au compte Lina et s'inscrire ;
5. revenir au compte club, actualiser l'événement et afficher la participante ;
6. annuler l'événement et constater son état ;
7. présenter les limites connues plutôt que d'actionner les commandes statiques du Feed.

Les tests Playwright exécutent automatiquement les variantes complètes, dont l'acceptation et le refus d'une demande, la capacité d'événement, les contrôles de rôle et l'affichage mobile.

Manuel de mise à jour et de maintenance de Spity - C2.4.1

Identification du document

Champ	Valeur
Produit	Spity
Version de référence	0.1.0
Public	Développeurs et responsables d'exploitation
Dépôt	Dorianyloj/spity
Branche d'intégration	develop
Branche stable	main
Historique	CHANGELOG.md et historique Git
Dernière vérification	20 juillet 2026

1. Objectif

Ce manuel décrit comment corriger, faire évoluer, mettre à niveau et publier Spity tout en conservant la traçabilité du code, du schéma de données, des tests et du déploiement. Il complète le [manuel de déploiement](#).

Une mise à jour n'est terminée que lorsque les sources, la migration éventuelle, les tests, le changelog, l'image déployée et le résultat de contrôle désignent la même révision Git.

2. Repères techniques et responsabilités

Zone	Technologie	Emplacement	Règle de maintenance
Pages et API	TypeScript, Next.js App Router, React	spity/src/app	Conserver les composants serveur par défaut ; valider les entrées et sorties API avec Zod.
Fonctionnalités	TypeScript strict	spity/src/features	Colocaliser composants, schémas, logique et dépôts par domaine. Aucun any explicite.
Interface	Tailwind CSS, composants internes, Lucide	spity/src/components/ui	Réutiliser le design system et vérifier clavier, contraste, mobile et lecteur d'écran.
Données	Drizzle ORM, MariaDB 11.4	spity/src/db , spity/drizzle	Générer une nouvelle migration ; ne jamais modifier une migration appliquée.
Contrats	Zod	schémas colocalisés	Parser les corps, paramètres et réponses ; retourner des erreurs contrôlées.
Tests	Jest, Node Test Runner, Playwright	tests colocalisés et spity/tests	Adapter les tests au même changement et préserver les seuils.

Zone	Technologie	Emplacement	Règle de maintenance
Exécution	Node.js 22, npm lockfile	<code>.nvmrc</code> , <code>package.json</code> , <code>package-lock.json</code>	Utiliser la version fixée et installer avec <code>npm ci</code> en validation.
Livraison	Docker, Compose, GitHub Actions, GHCR	<code>Dockerfile</code> , <code>.github/workflows</code>	Construire une image par SHA, promouvoir le même candidat et ne jamais remplacer un tag stable manuellement.

Le choix de TypeScript strict, Zod et Drizzle fournit trois contrôles complémentaires : compilation, validation à l'exécution et schéma relationnel versionné. Docker et le lockfile réduisent les écarts entre poste, CI et production. Git et le changelog assurent la traçabilité humaine de chaque évolution.

3. Classification et version

Spity suit Semantic Versioning `MAJEUR.MINEUR.CORRECTIF` :

- **correctif** : correction rétrocompatible, par exemple `0.1.0` vers `0.1.1` ;
- **mineur** : nouvelle fonction rétrocompatible, par exemple `0.1.0` vers `0.2.0` ;
- **majeur** : rupture d'API, de données ou de procédure après la version `1.0.0`.

Chaque commit suit Conventional Commits : `feat`, `fix`, `docs`, `test`, `refactor`, `perf`, `style` ou `chore`, avec un scope utile. Les changements notables sont ajoutés sous `Unreleased` dans `CHANGELOG.md`.

Avant de commencer, qualifier :

1. le besoin ou l'anomalie et son impact ;
2. les fonctions et contrats concernés ;
3. la nécessité d'une migration ;
4. les risques sécurité, accessibilité et perte de données ;
5. les tests et le niveau de version attendus ;
6. le plan de retour arrière.

4. Flux standard de modification

Depuis la racine du dépôt :

```
git switch develop
git pull --ff-only
git switch -c feat/description-courte
cd spity
npm ci
cp .env.example .env.local
docker compose --env-file .env.local up -d mariadb
npm run db:migrate
```

Si la branche corrige une anomalie, utiliser `fix/description-courte`. Ne pas pousser directement sur `main`.

Pendant le développement :

- conserver TypeScript strict et les alias `@/` ;
- placer la logique métier dans la feature concernée plutôt que dans la page ;
- parser toute donnée externe comme `unknown` avec Zod ;

- ne pas consigner de secret, mot de passe, cookie ni donnée personnelle ;
- ajouter ou adapter les tests au comportement modifié ;
- mettre à jour les trois manuels si la procédure ou l'interface change.

Avant chaque commit, examiner uniquement ses changements :

```
git status --short
git diff --check
git diff
```

Créer un commit logique, pousser la branche, puis ouvrir une pull request vers `develop`. La revue vérifie le besoin, le risque, les tests, la migration, l'accessibilité, la sécurité et la documentation.

5. Modifier la base de données

Le schéma source est [spity/src/db/schema.ts](#). Les migrations générées se trouvent dans [spity/drizzle](#).

5.1 Créer une migration

1. Modifier le schéma Drizzle.
2. Générer un nouveau fichier :

```
cd spity
npm run db:generate
```

3. Relire le SQL généré et les métadonnées associées.
4. Vérifier qu'aucune migration existante n'a été réécrite.
5. Appliquer sur une base locale jetable ou sauvegardée :

```
npm run db:migrate
```

6. Lancer les tests d'intégration puis la recette des parcours concernés.

`npm run db:push` ne doit pas être utilisé pour une évolution partagée ou de production, car il ne produit pas l'historique de migration attendu.

5.2 Stratégie sans interruption

Préférer une séquence **étendre, migrer, contracter** :

1. ajouter une colonne ou table compatible avec l'ancienne application ;
2. déployer le code capable de lire ancien et nouveau formats si nécessaire ;
3. migrer ou compléter les données ;
4. observer la version ;
5. supprimer l'ancien champ dans une release ultérieure seulement.

Une migration destructive et son nettoyage ne doivent pas être regroupés avec l'introduction du nouveau modèle. Cette séparation maintient un retour arrière applicatif possible.

5.3 Correction d'une migration déjà appliquée

Ne jamais éditer son fichier. Créer une migration supplémentaire qui corrige l'état. Documenter la cause et le retest dans le [registre des bogues](#).

6. Mettre à jour une dépendance

Examiner d'abord l'état sans modifier le lockfile :

```
cd spity
npm outdated
npm audit --omit=dev
```

Procédure :

1. lire la note de version et le guide de migration de la source officielle ;
2. vérifier la compatibilité Node.js 22, Next.js, React et TypeScript ;
3. mettre à jour une famille cohérente de dépendances à la fois ;
4. utiliser `npm install nom@version` pour actualiser ensemble `package.json` et `package-lock.json` ;
5. adapter le code et les tests sans désactiver une règle de qualité ;
6. exécuter tous les contrôles concernés, puis `npm audit --omit=dev --audit-level=high` ;
7. consigner la mise à jour et toute rupture dans le changelog.

Pour changer la version majeure de Node.js, modifier de façon cohérente `.nvmrc`, `package.json#engines`, le `Dockerfile` et les workflows. Vérifier que les actions GitHub utilisent leur runtime maintenu ; les workflows actuels emploient les versions `v6`, exécutées nativement sur Node.js 24 côté runner.

Ne jamais lancer une mise à jour globale non relue ni supprimer le lockfile pour résoudre un conflit. Une alerte de sécurité critique ou élevée bloque la publication jusqu'à correction, remplacement ou décision de risque documentée.

7. Validation avant intégration

7.1 Portes locales minimales

```
cd spity
npm run lint
npm run typecheck
npm run test:coverage
npm run security:audit
npm run build
```

Seuils unitaires actuels : 80 % lignes, fonctions et instructions, 75 % branches sur le périmètre Jest. Une baisse de seuil ne constitue pas une correction.

7.2 Contrôles avec MariaDB et navigateur

Avec `DATABASE_URL` pointant vers une base isolée :

```
npm run db:migrate
npm run test:integration
npm run test:acceptance
npm run accessibility:audit
npm run perf:audit
```

Installer Chromium au besoin avec `npx playwright install chromium`. Les scripts navigateur démarrent leurs propres serveurs ; ne pas réutiliser le même port pour plusieurs audits simultanés.

Le périmètre et les preuves sont détaillés dans le [harnais unitaire](#), les [tests d'intégration](#), les audits [OWASP](#) et [RGAA](#), ainsi que le [cahier de recettes](#).

7.3 CI obligatoire

La pull request puis `develop` exécutent les portes distantes. Le staging continu n'est produit qu'après succès du lint, typage, tests, audit des dépendances, build, intégration MariaDB, accessibilité, performance et recette Playwright.

Ne pas fusionner avec un job obligatoire rouge. Une relance est acceptable uniquement après analyse d'une cause transitoire ; elle ne remplace pas une correction.

8. Préparer et publier une version

8.1 Préparer la révision candidate

1. Vérifier que tous les changements prévus sont intégrés dans `develop`.
2. Choisir la nouvelle version SemVer.
3. Mettre à jour sans créer automatiquement de tag :

```
cd spity
npm version --no-git-tag-version 0.1.1
cd ..
```

4. Déplacer les éléments pertinents de `Unreleased` vers `## [0.1.1] - AAAA-MM-JJ` et mettre à jour les liens du changelog.
5. Exécuter la validation complète.
6. Committer `package.json`, `package-lock.json` et `CHANGELOG.md`, puis pousser `develop`.
7. Attendre que le staging continu de ce SHA précis soit vert et que ses images `sha-SHA_GIT` existent dans GHCR.

8.2 Créer le tag

Le tag doit viser exactement le commit passé par le staging continu :

```
git status --short
git tag --annotate v0.1.1 --message "Spity v0.1.1"
git push origin v0.1.1
```

Ne jamais déplacer ni forcer un tag stable. Le workflow de release :

1. vérifie la concordance tag, `package.json` et changelog ;
2. rejoue les portes qualité, les migrations, l'intégration, l'accessibilité et Lighthouse ;
3. récupère les images immuables `sha-SHA_GIT` issues du staging ;

4. exécute migration et smoke test dans un environnement isolé ;
5. promeut les mêmes digests sous `candidate-VERSION`, `VERSION` et `latest` ;
6. génère le bundle, le manifeste et la somme SHA-256 ;
7. publie la release GitHub.

La recette Playwright ne se répète pas dans le workflow de tag : elle a déjà bloqué la construction staging du SHA immuable que la release exige et promeut. La validation de tag exécute en complément ses propres contrôles et smoke tests.

8.3 Vérifier les livrables

Télécharger le bundle et contrôler :

```
sha256sum --check spity-0.1.1.tar.gz.sha256
tar --list --gzip --file spity-0.1.1.tar.gz
```

L'archive doit contenir le Compose de production, le modèle d'environnement, `DEPLOYMENT.md`, le changelog et `release-manifest.json`. Comparer les digests du manifeste avec GHCR avant déploiement.

9. Mise à jour d'une instance déployée

Suivre dans l'ordre le [manuel de déploiement](#) :

1. conserver l'ancienne version, son SHA et ses digests ;
2. vérifier le nouveau bundle et préparer `.env.production` ;
3. tirer les images versionnées ;
4. attendre MariaDB saine et créer une sauvegarde externalisée ;
5. exécuter le conteneur de migration ponctuel ;
6. démarrer l'application sans reconstruction locale ;
7. comparer `/api/health` à la version et au SHA attendus ;
8. exécuter les contrôles fonctionnels HTTPS ;
9. consigner le résultat et conserver les preuves.

Le déploiement ne doit pas utiliser un `docker compose build` improvisé sur le serveur : l'image publiée et testée est la référence.

10. Retour arrière d'une mise à jour

Déclencher le retour arrière si la santé échoue, si la version est incohérente, si une fonction critique régresse ou si les erreurs affectent les utilisateurs.

1. noter l'incident et conserver les journaux ;
2. remettre `IMAGE_TAG`, `APP_VERSION` et `APP_REVISION` de la dernière version validée ;
3. relancer l'image applicative précédente ;
4. vérifier santé et parcours critiques ;
5. restaurer la base uniquement si la migration empêche réellement ce retour applicatif ;
6. créer une anomalie avec cause, impact, décision et résultat du retest.

La commande de restauration et ses avertissements figurent dans le [manuel de déploiement, section 9](#).

11. Maintenance courante

Fréquence indicative	Contrôle	Trace attendue
À chaque déploiement	Sauvegarde, migration, santé, version, parcours critiques	Journal de déploiement et manifeste.
Hebdomadaire	Disponibilité HTTPS, santé, redémarrages, erreurs applicatives et espace disque	Relevé d'exploitation sans secret.
Mensuelle	<code>npm outdated</code> , audit de dépendances, images de base et actions CI	Ticket de mise à jour ou acceptation de risque datée.
Trimestrielle	Test de restauration sur environnement isolé, revue des accès GHCR et secrets	Procès-verbal de restauration et revue d'accès.
Avant soutenance ou release	Recette F01-F10, accessibilité, Lighthouse et contrôle mobile	Rapports CI et captures datées.

Renouveler un secret compromis immédiatement, invalider les accès associés puis redéploier. Ne pas publier l'ancienne valeur dans le ticket d'incident.

12. Traçabilité d'une intervention

Pour chaque mise à jour, conserver :

Élément	Exemple
Besoin ou anomalie	Identifiant, scénario, résultat attendu et observé.
Source	Branche, pull request, commits et auteur Git.
Version	Ancienne et nouvelle versions SemVer.
Données	Migration ajoutée, résultat et sauvegarde associée.
Validation	Commandes, rapports, URL d'exécution CI et décisions de revue.
Artefact	Tag, SHA, images et digests du manifeste.
Exploitation	Heure, opérateur, santé, contrôles fonctionnels et retour arrière éventuel.

Cette chaîne relie une décision métier à la version effectivement exploitée et rend les évolutions futures compréhensibles par une autre équipe.

13. Checklist de maintenance

- ☐ Besoin, risque, version et plan de retour arrière qualifiés.
- ☐ Branche dédiée et historique Git propre.
- ☐ Contrats Zod, code strict et tests mis à jour.
- ☐ Nouvelle migration relue si le schéma change ; aucune ancienne migration modifiée.
- ☐ Changelog et documentation d'exploitation actualisés.
- ☐ Portes locales et CI réussies sans abaissement des seuils.
- ☐ SHA candidat réellement promu par le staging.

- ☐ Tag immuable, bundle, somme et manifeste vérifiés.
- ☐ Sauvegarde et contrôles post-déploiement consignés.

Clôture du livrable

Ce document rassemble la synthèse, l'index exhaustif des critères et les quatorze livrables détaillés du bloc BC02. Les empreintes des sources, du HTML et du PDF sont conservées dans le manifeste joint.

Candidat	Dorian Joly
Projet	Spity 0.1.0
Bloc	BC02 · RNCP39583
Document	DOSSIER_BC02_SPITY.pdf